

Aula 02

Ciência de Dados



Uni-FACEF

**Departamento de
Computação**



CIÊNCIA DE DADOS

- *O que é ciência de dados?*
 - A **CD** é uma área de conhecimento que estuda e aplica **princípios** e **técnicas**, implementados por meio de algoritmos, para extrair conhecimento novo, relevante e útil de um conjunto de dados
 - Busca respostas para a seguinte pergunta: **como extrair** (eficientemente) **conhecimento** em (grandes) **conjuntos** (de fluxos) de dados que apoie o processo de tomada de decisão?

Introdução

- ***Etapas de um Projeto de CD***

- A **execução** de um **projeto de CD** ocorre por meio de um **processo** ou **pipeline**, formado por uma **sequência de etapas de processamento**
- Uma das **primeiras propostas** para **sistematizar** todo o **processo de extração de conhecimento de conjunto de dados** foi a área de **descoberta de conhecimento em bancos de dados** (**KDD**, **knowledge discovery in database**)



- ***Etapas de um Projeto de CD***
 - ***Processo iterativo e interativo, formado por uma sequência de novo passos:***
 1. ***Entender o domínio da aplicação e conhecimento prévio relevante, além de identificar o objetivo do processo de ponto de vista do cliente***
 2. ***Selecionar um conjunto de dados, uma amostra de um conjunto ou um subconjunto de atributos, descoberta***

- ***Etapas de um Projeto de CD***
 - ***Processo iterativo e interativo, formado por uma sequência de novo passos:***
 3. ***Limpar o conjunto de dados, melhorando sua qualidade, e pré-processar os dados que pode incluir a identificação e remoção de **ruídos**, o tratamento de **valores ausentes** em atributos, e o manejo para lidar com aspectos temporais***
 4. ***Reduzir a dimensionalidade dos dados, projetando ou selecionando os atributos mais relevantes para a tarefa a ser realizada***

- ***Etapas de um Projeto de CD***
 - ***Processo iterativo e interativo, formado por uma sequência de novo passos:***
 5. ***Conciliar o objetivo do processo **KDD** com a tarefa de Mineração de Dados (MD), (classificação, agrupamento, sumarização, etc.)***
 6. ***Escolher o(s) algoritmo(s) a ser(em) utilizado(s), ajustar o valor de hiper-parâmetros, selecionar modelos gerados pelo(s) algoritmo(s) e conciliar os critérios usados na seleção de modelos com o objetivo do cliente (melhor desempenho preditivo, menor custo computacional, etc.)***

- ***Etapas de um Projeto de CD***
 - ***Processo iterativo e interativo, formado por uma sequência de novo passos:***
 - 7. Minerar os dados, procurando por padrões de interesse nos modelos gerados, conforme a representação do modelo e a tarefa de MD***
 - 8. Interpretar os padrões minerados, por meio de técnicas de visualização, podendo retornar a qualquer um dos passos anteriores***

- ***Etapas de um Projeto de CD***
 - ***Processo iterativo e interativo, formado por uma sequência de novo passos:***
 9. ***Agir utilizando o conhecimento descoberto, incorporando-o em novos sistemas para ações posteriores, ou apenas documentá-lo e comunicá-lo ao cliente, além de buscar e resolver possíveis conflitos de interesse com algum conhecimento anterior***

Introdução

- **Etapas de um Projeto de CD**

Pré-Processamento:

Pode consumir 60% ou mais tempo no processo de descoberta de conhecimento

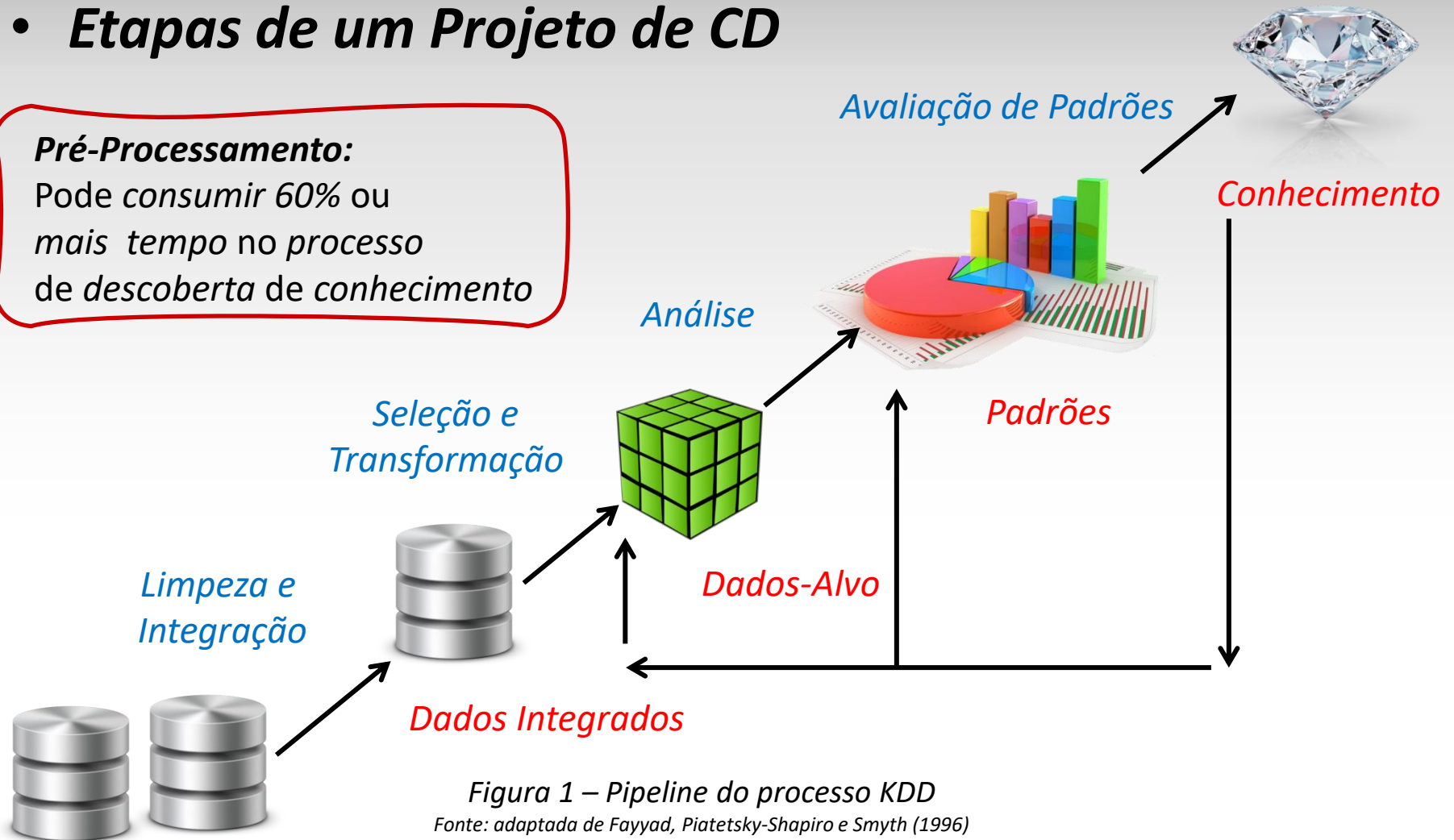


Figura 1 – Pipeline do processo KDD

Fonte: adaptada de Fayyad, Piatetsky-Shapiro e Smyth (1996)

- ***Etapas de um Projeto de CD***
 - ***A metodologia **CRISP-DM** desenvolveu um **novo fluxo** de processo para descoberta de conhecimento, a partir do processo de KDD, para responder às demandas das indústrias***
 - ***A metodologia tinha por meta tornar os projetos:***
 - ***mais rápidos***
 - ***mais baratos***
 - ***mais confiáveis***
 - ***mais facilmente gerenciáveis***

- **Metodologia CRISP-DM**

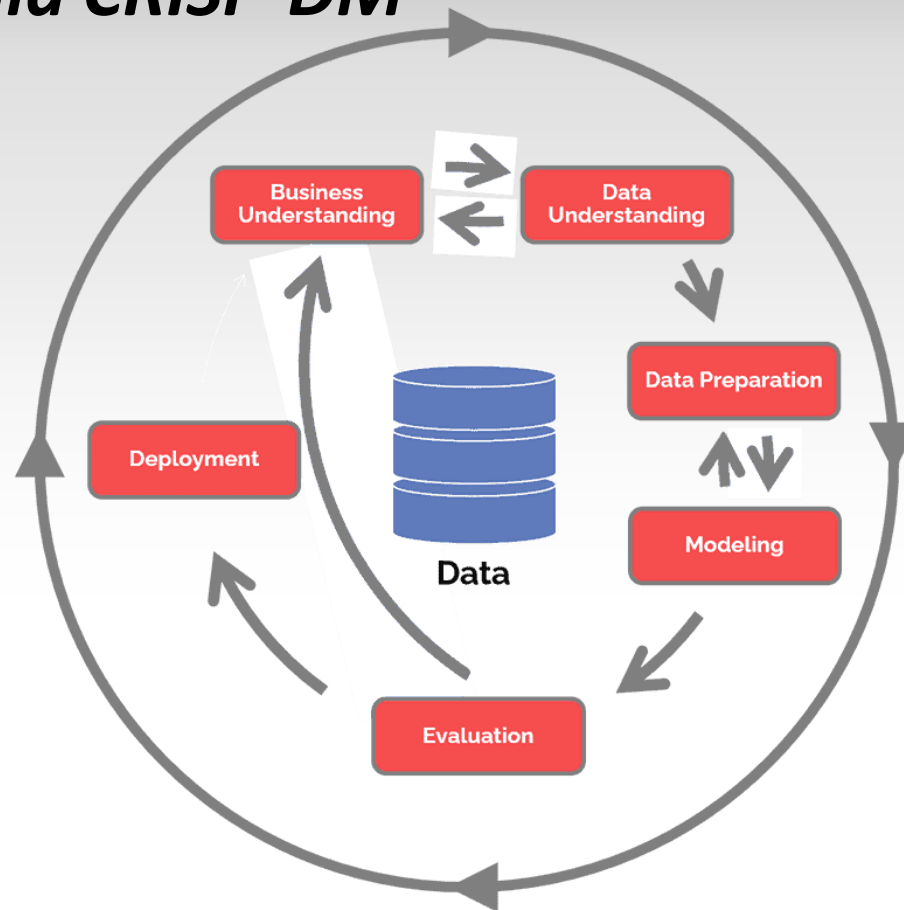


Figura 1 – Pipeline da metodologia CRISP-DM

Fonte: adaptada de Charpman et al. (2000)

- ***Etapas de um Projeto de CD***
 - ***Sumarização das principais diferenças entre MD e CD***

Mineração de Dados	Ciência de Dados
Área de conhecimento, que surgiu a partir da área de BD, parte do processo de KDD	Área multidisciplinar, que surgiu da combinação de conceitos da computação, da estatística e da matemática
Abrange a descoberta de padrões em dados	Abrange praticamente tudo que diz respeito à AD
Utiliza modelos gerados para extrair informações de um conjunto de dados	Inclui coleta, dados, melhoria da qualidade dos dados, transformação de dados, pré-processamento de dados, indução e aplicação de modelos, visualização de dados, modelos e resultados, análise e validação do conhecimento extraído, e uso do conhecimento

Tabela 1 – Principais diferenças entre Mineração de Dados e Ciência de Dados

- ***Etapas de um Projeto de CD***
 - ***Sumarização das principais diferenças entre MD e CD***

Mineração de Dados	Ciência de Dados
Trabalha com dados estruturados	Trabalha com dados estruturados, semiestruturados e não estruturados
Surgiu de demandas empresariais	Surgiu de demandas científicas
Maior foco no processo do negócio	Maior foco na ciência dos dados
Orientada a objetivos, com foco no processo de negócio	Orientada a dados, com foco na exploração de dados
Orientada a objetivos	Orientada a objetivos, orientada à exploração de dados ou orientada a gerenciamento de dados
É abrangida pela CD	Inclui a Mineração de Dados

Tabela 1 – Principais diferenças entre Mineração de Dados e Ciência de Dados

Introdução

- ***Etapas de um Projeto de CD***
 - ***Sumarização das principais diferenças entre MD e CD***

Mineração de Dados	Ciência de Dados
Trabalha com dados estruturados	Trabalha com dados estruturados, semiestruturados e não estruturados
Sur	Com a CD, os projetos se tornam mais exploratórios , levando a uma maior variação de pipeline, precisando de uma metodologia mais flexível, baseada em trajetórias em vez de processos e execução de diferentes tarefas
M	
Or	
pr	
Orientada a objetivos	Orientada a objetivos, orientada à exploração de dados ou orientada a gerenciamento de dados
É abrangida pela CD	Inclui a Mineração de Dados

Tabela 1 – Principais diferenças entre Mineração de Dados e Ciência de Dados

- ***Ciclo de Vida dos Dados***

- ***De acordo com Berman et al. (2018), dados, assim como organismos biológicos, possuem um ciclo de vida, que vai do nascimento a uma vida ativa***
- ***No entanto, dados podem tanto ser removidos (morrer) ou permanecerem vivos para sempre, atingindo a imortalidade***
- ***Similarmente aos seres vivos, dados sobrevivem se estiverem em um ambiente que fornece suporte físico, contexto social e um sentido para existir***

- ***Ciclo de Vida dos Dados***

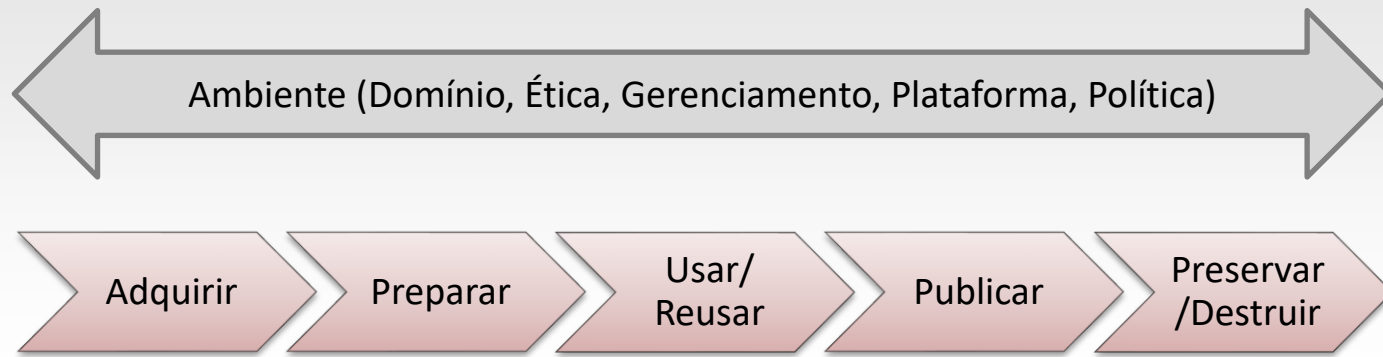


Figura 2 – Ciclo de Vida dos Dados

Fonte: adaptada de Berman et al. (2018)

Introdução

- *Lago de Dados*

- É um **repositório de dados** que **propicia** o **armazenamento**, **manutenção** e **disponibilização** de uma **grande quantidade** e **variedade** de dados “crus”, dados em seus formatos originais



- ***Lago de Dados***

- ***A organização de dados em lagos possui vários benefícios, dentre os quais se destacam:***
 - ***Permite o acesso e a exploração desses dados por vários usuários de diferentes necessidades ou demandas***
 - ***Facilita a integração das fontes, que exercem, papel semelhante ao de afluentes que alimentam um lago, de dados de uma organização***
 - ***Favorece a captura, a mistura e a exploração de novos tipos de dados, possibilitando a extração conhecimentos novos e de maior valor***

Introdução

- ***Lago de Dados***

- *Uma estrutura maior e mais sofisticada para coletar, armazenar, manter e disponibilizar dados, como os lagos de dados, precisa ser **bem gerenciada**, sob risco de que os dados **não** sejam encontrados ou que o lago se torne um depósito de **entulho** ou **poluído** (**governança de dados**)*



- ***Governança de Dados***

- ***É uma estratégia, com um conjunto de práticas, para gerenciamento de dados, garantindo a **qualidade**, a **integridade**, a **segurança** e a **usabilidade** dos dados coletados ou gerados por uma organização***
- ***Deve ocorrer desde o momento em que o dado é coletado ou gerado até o momento em que os dados são destruídos ou removidos para serem arquivados***



- ***Governança de Dados***
 - ***Deve seguir um conjunto de boas práticas (códigos de comportamento) e processos para que a manipulação de dados ocorra de uma forma adequada (padrão)***
 - ***Ela tem por meta garantir que os dados sejam:***
 - ***acessíveis para quem deles precisar e tem permissão***
 - ***confiáveis***
 - ***documentados***
 - ***seguros***

- ***Governança de Dados***
 - ***Define*** quais são os ***direitos*** e as ***responsabilidades*** de quem ***realizará ações sobre:***
 - ***quais dados***
 - ***de que forma***
 - ***quando***
 - ***em que situação ou circunstâncias***
 - ***utilizando quais métodos***

- ***Governança de Dados***

- *O modelo de governança adotado deve definir **regulações** e **políticas** sobre como essas ações devem ser realizadas*
- *Um bom modelo de governança deve garantir que os dados sejam **confiáveis** e que as pessoas que realizam ações que **usem** ou **afetem** os dados sejam responsabilizadas*
- *A governança pode incluir ainda aspectos **tecnológicos**, relacionados com a **infraestrutura** utilizada, as **ferramentas** utilizadas e disponibilizadas ao ciclo de vida dos dados*

- ***Governança de Dados***

- *O responsável pela governança de dados é geralmente chamado de gerente de governança de dados (data stewards)*
- *Deve vir acompanhada de uma atividade de curadoria*
- *A tarefa de governança de dados é semelhante à tarefa de um administrador ou gestor*



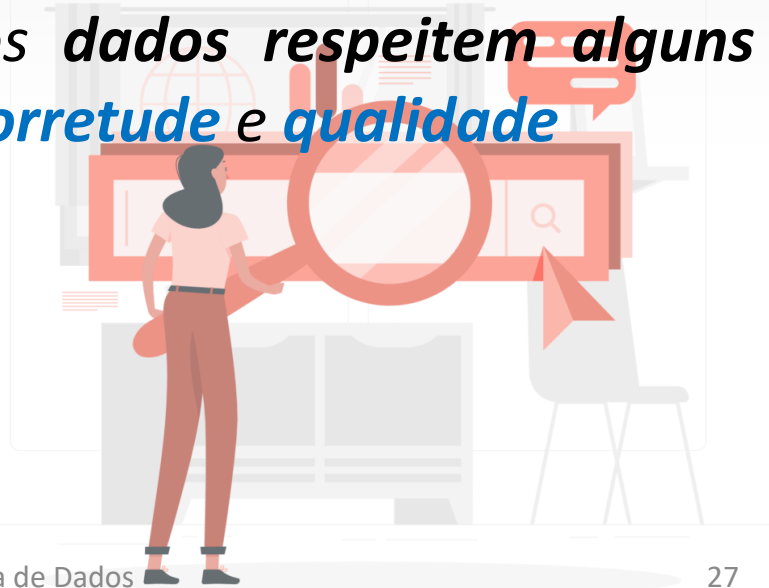
- ***Curadoria de Dados***

- *Visa aperfeiçoar os metadados e a sua qualidade, facilitando sua **preservação** e **exploração***
- *Tem similaridades com a curadoria de arte em museus e galerias, em que **objetos encontrados na natureza**, como **animais**, **plantas** e **pedras**, ou **objetos criados por seres humanos**, como **quadros**, **esculturas**, ou **tecnologias**, são **documentados** (cada obra é documentada com informações relacionadas com a procedência, natureza, entre outros)*

Introdução

- **Curadoria de Dados**

- Uma das **principais atividades** do **ciclo de vida** de **CD** é a **curadoria** e a **governança** de **dados**
- Os bens mais valiosos da **CD** são os **conjuntos** de **dados**
- Como qualquer produto valioso, é importante conhecer a **procedência**, e **garantir** que os **dados** respeitem alguns princípios relacionados com a **corretude** e **qualidade**



- ***Curadoria de Dados***
 - ***As principais tarefas de curadoria de dados são:***
 - ***autenticação***
 - ***elaboração e manutenção de cadeia de custódia***
 - ***documentação***
 - ***validação do documento***
 - ***criação de metadados***

- ***Curadoria de Dados***

- ***Autenticação***

- ***Confirma a identidade de uma pessoa, em geral, quem armazenou e está contribuindo com os dados para o repositório***
 - ***Pode ser feito por meio de senha (password) ou autorização via assinatura digital***
 - ***É essencial para seguir a precedência dos dados***



- ***Curadoria de Dados***
 - ***Cadeia de Custódia***
 - ***Define os metadados com o registro da proveniência dos arquivos***
 - ***Inclui quem criou, quando foi editado pela última vez, quem editou, além de outras informações que permitem preservar a autenticidade do arquivo quando os dados são transferidos para terceiro***

- ***Curadoria de Dados***

- ***Documentação***

- ***Produção de um documento com as informações necessárias para utilizar e entender os dados***
 - ***Pode ser um arquivo `readme`, no formato `.txt`, ou documento com um formato mais elaborado, que pode incluir gráficos e tabelas***



- ***Curadoria de Dados***

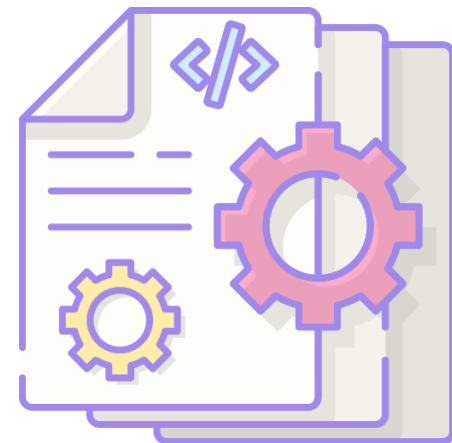
- ***Validação do Documento***

- ***É um processo computacional, o qual garante que a transferência do arquivo para o repositório ocorreu perfeitamente***
 - ***Pode ser feita pela verificação de arquivo quando ele pode ser analisado, por exemplo, se o arquivo recebido tem o mesmo número de bits do arquivo original, ou pela verificação de formato, que pode checar se o tipo ou conteúdo do arquivo corresponde à sua extensão***

- **Curadoria de Dados**

- **Metadados**

- **São informações sobre o conjunto de dados que podem ser usados para *busca* e *recuperação*, e devem estar em formato que possa ser lido computacionalmente e incluir informações básicas (título, autor, data da criação, período coberto, número de exemplos, número de atributos preditivos e distribuição dos dados)**



- ***Curadoria de Dados***

- *Em um processo de **curadoria**, conjuntos de dados são **organizados, descritos, limpos, melhorados e preservados** para uso público*
- *Os meios de armazenamento disponibilizados pela internet permitem **postar e compartilhar dados de forma fácil e segura***
- *Sem curadoria, os dados podem ser **difíceis** de **encontrar, usar e interpretar***

- ***Trabalho em Equipe***

- ***Dada a sua complexidade, projetos de sistemas baseados em CD são geralmente conduzidos por equipes, cujos participantes têm formações e interesses complementares***
- ***Para que o trabalho em equipe seja bem-sucedido, foi publicado um manifesto chamado “Valores e Princípios dos Dados”, com 4 valores e 12 princípios para trabalhos em equipe***

- ***Trabalho em Equipe***

- ***Os valores defendidos são:***

- 1. Inclusão, maximizando diversidade, conectividade e acessibilidade entre projetos, colaboradores e resultados***
- 2. Experimentação, enfatizando o teste e a AD de modo contínuo e interativo***
- 3. Responsabilização, por meio de um comportamento ético e transparente, corrigindo erros rapidamente e atribuindo definições para quem for responsável***

- ***Trabalho em Equipe***

- ***Os valores defendidos são:***

- 4. Impacto, priorizando projetos com objetivos bem definidos e os direcionando para atingir resultados mensuráveis e substantivos***



- ***Trabalho em Equipe***

- ***Os princípios a serem adotados são:***

- 1. Utilizar dados para melhorar a vida de usuários, clientes, organizações e comunidade***
- 2. Criar um trabalho reproduzível e extensível***
- 3. Organizar equipes com diversas ideias, formações e pontos fortes***
- 4. Priorizar a coleta contínua e a disponibilidade de discussões e metadados***

- ***Trabalho em Equipe***

- ***Os princípios a serem adotados são:***

- 5. Identificar claramente as perguntas a serem respondidas e os objetivos que impulsionam cada projeto e utilizá-los para orientar o planejamento e o refinamento***
- 6. Estar aberto a mudar métodos e conclusões em respostas a novos conhecimentos***
- 7. Reconhecer e reduzir preconceitos pessoais e nos dados utilizados***

- ***Trabalho em Equipe***

- ***Os princípios a serem adotados são:***

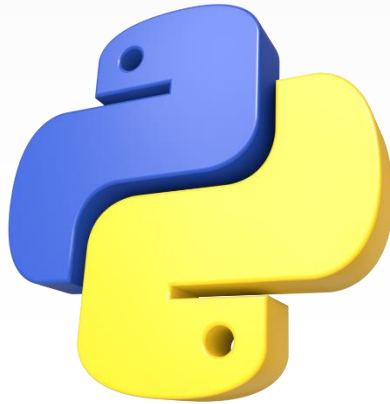
- 8. Apresentar o trabalho para capacitar outras pessoas a tomarem decisões mais bem informadas***
- 9. Considerar cuidadosamente as implicações éticas das escolhas feitas ao utilizar dados e os impactos do trabalho nos indivíduos e na sociedade***
- 10. Estimular e respeitar críticas justas, promovendo a identificação e discussão aberta de erros, riscos e consequências não intencionais do trabalho***

- ***Trabalho em Equipe***

- ***Os princípios a serem adotados são:***

- 11. Proteger a privacidade e a segurança dos indivíduos representados nos dados***

- 12. Ajudar outras pessoas a entender as mais úteis e adequadas aplicações de dados para resolução de problemas reais***



INTRODUÇÃO AO PYTHON

Introdução

- **Python**

- *Linguagem de programação de **alto nível** e de propósito geral, criada com o objetivo de ser uma linguagem fácil para implementação de algoritmos, possuindo uma **sintaxe simples e intuitiva***
- *Amplamente utilizada em **diversas áreas**, incluindo **desenvolvimento web**, **ciência de dados** e **inteligência artificial***



- **Python**

- *Sintaxe é bastante clara e intuitiva, seguindo algumas convenções comuns a muitas outras linguagens de programação, como:*

- **Indentação**

- *É chamado de indentação o recuo à direita no início das linhas do código, serve para indicar o início e o final de um bloco de código*
 - *O Python usa indentação, em vez de usar chaves ou outros delimitadores (a indentação deve ser consistente em todo o código, geralmente com quatro espaços)*

- ***Python***

- ***Sintaxe é bastante clara e intuitiva, seguindo algumas convenções comuns a muitas outras linguagens de programação, como:***

- ***Comentários***

- ***São iniciados por "*#*"* e continuam até o fim da linha**
 - ***Não existe um método padrão para comentar várias linhas de código em Python***

- **Python**

- *Sintaxe é bastante clara e intuitiva, seguindo algumas convenções comuns a muitas outras linguagens de programação, como:*

- **Variáveis**

- *São criadas assim que são atribuídas um valor, não precisam ser declaradas explícitas com tipos e sua atribuição é feita com o sinal "="*
 - *Tipos explícitos: int, float, complex, object (strings), bool, list, tuple e dict*

- **Python**

- *Sintaxe é bastante clara e intuitiva, seguindo algumas convenções comuns a muitas outras linguagens de programação, como:*

- **Operadores**

- *Suporta os operadores matemáticos comuns, incluindo **+**, **-**, *****, **/**, **%** (módulo), ****** (*potência*) e muitos outros*

- **Controle de Fluxo**

- *Suporta as estruturas de controle de fluxo comuns, incluindo **if**, **elif**, **else**, **for** e **while***

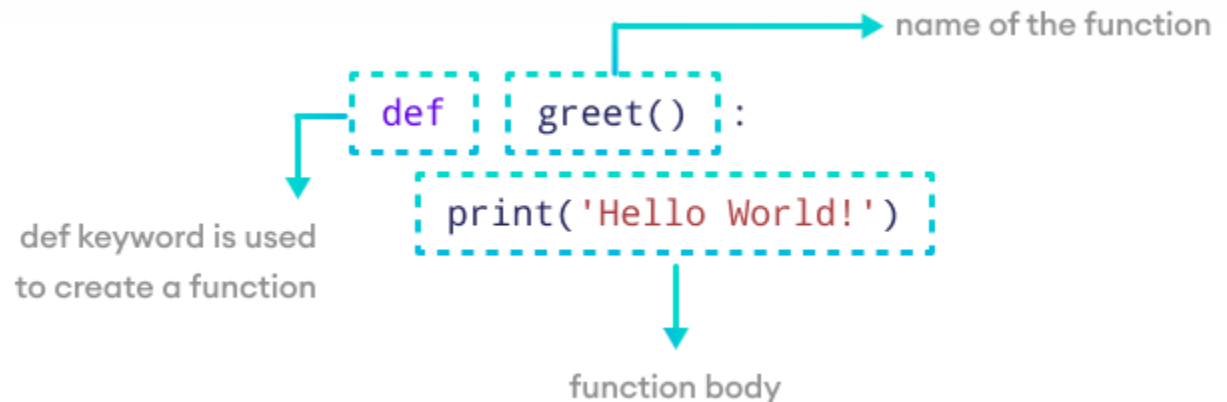
Introdução

- **Python**

- *Sintaxe é bastante clara e intuitiva, seguindo algumas convenções comuns a muitas outras linguagens de programação, como:*

- **Funções**

- *São definidas usando a palavra-chave "**def**", seguida pelo nome da função e parênteses*



- **Regras para nomes de Variáveis**
 - Os nomes de variáveis devem obedecer algumas **regras simples**:
 - Devem sempre começar com uma letra minúscula ou underline
 - Deve ser uma sequência de letras ($a \rightarrow z$, $A \rightarrow Z$) e números ($0 \rightarrow 9$)
 - Apenas letras comuns são permitidas
 - **Não** são permitidos acentuação, cedilhas, espaços, caracteres especiais (\$, #, @, !, %, entre outros)

- ***Regras para nomes de Variáveis***
 - Os ***nomes de variáveis*** devem obedecer algumas ***regras simples***:
 - ***Pode ser usado o caractere _ (sublinhado/underline)***
 - ***Palavras reservadas da linguagem não são permitidas (print, True, False, if, else, min, max, entre outras)***



- **Jupyter Notebook**

- É uma **ferramenta** de **desenvolvimento interativo** que se tornou fundamental no campo da **ciência de dados**, **análise estatística**, e **educação em programação**
- Desenvolvido originalmente como parte do projeto **IPython**, o Jupyter é uma **aplicação web de código aberto** que **permite a criação e compartilhamento** de documentos contendo **código**, **visualizações** e **texto explicativo**



- *Jupyter Notebook*

- *Suportar múltiplas linguagens de programação, incluindo **Python**, **R**, **Julia** e muitas outras*
- *É baseado em **células**, onde cada célula pode conter **código** executável, **texto formatado** usando a linguagem de marcação **Markdown**, ou **visualizações interativas***
- *Facilita a criação de documentos que mesclam código, resultados computacionais e explicações em um formato claro e compreensível*

Introdução

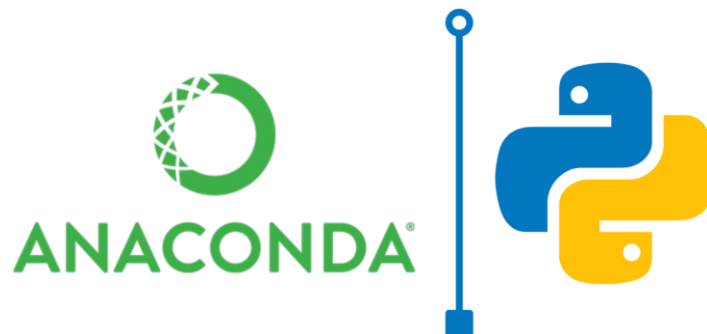
- *Jupyter Notebook*

- *A capacidade de exportar notebooks para diferentes formatos, como **HTML**, **PDF** ou **slides** de **apresentação**, torna o Jupyter uma ferramenta versátil para compartilhar e comunicar resultados de análises complexas*



- **Anaconda**

- *Plataforma de código aberto projetada para simplificar e acelerar o processo de desenvolvimento, distribuição e gerenciamento de aplicativos e ambientes de ciência de dados*
- *Tornou-se uma escolha popular entre os profissionais que trabalham com análise de dados, aprendizado de máquina e computação científica*



- **Anaconda**
 - Inclui um gerenciador de pacotes chamado **Conda**
 - Facilita a **instalação**, **atualização** e **remoção** de pacotes, garantindo que as **dependências** sejam gerenciadas de maneira eficiente
 - **Crucial** quando se lida com um **ecossistema complexo** de **bibliotecas** e **ferramentas** que são frequentemente utilizadas em projetos de ciência de dados

Introdução

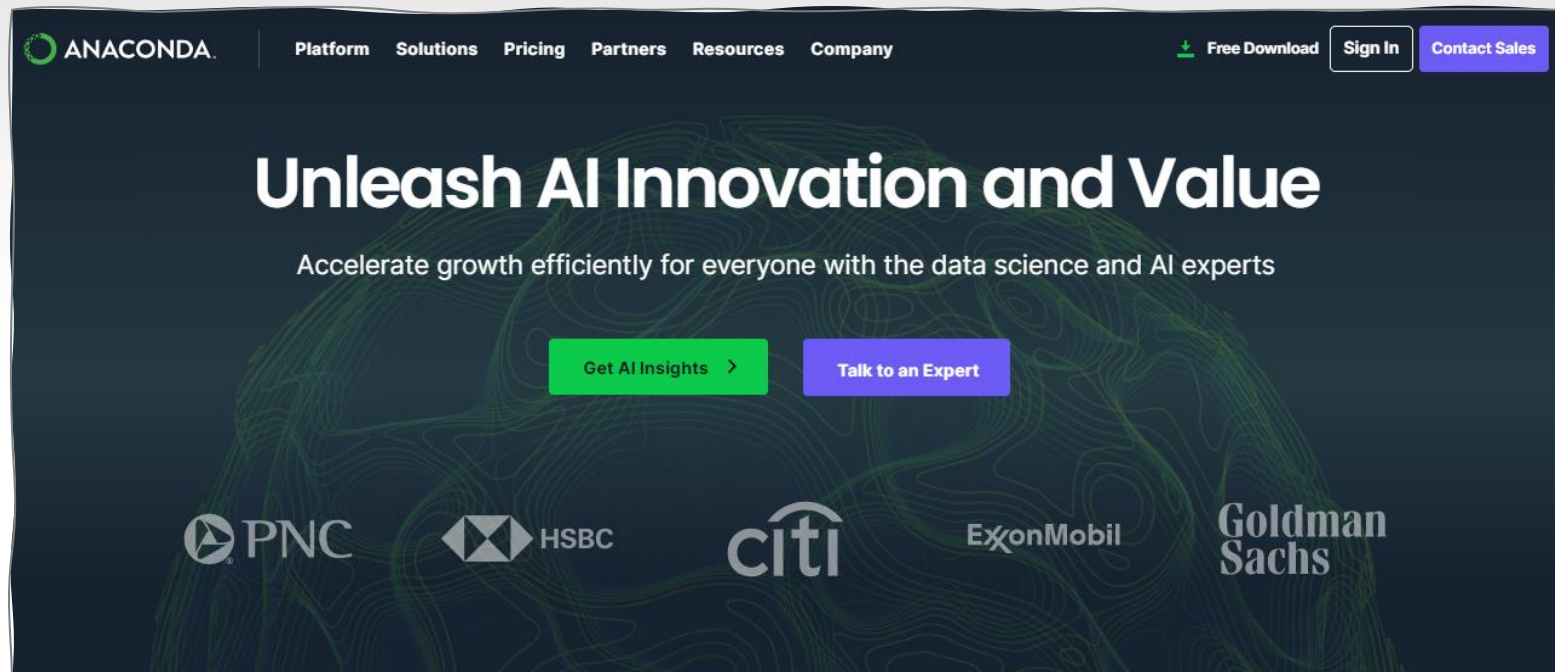
- **Anaconda**

- *Inclui uma extensa coleção de pacotes pré-instalados, otimizados para trabalhar em conjunto sem problemas (inclui bibliotecas populares como NumPy, pandas, Matplotlib, SciPy e muitas outras)*

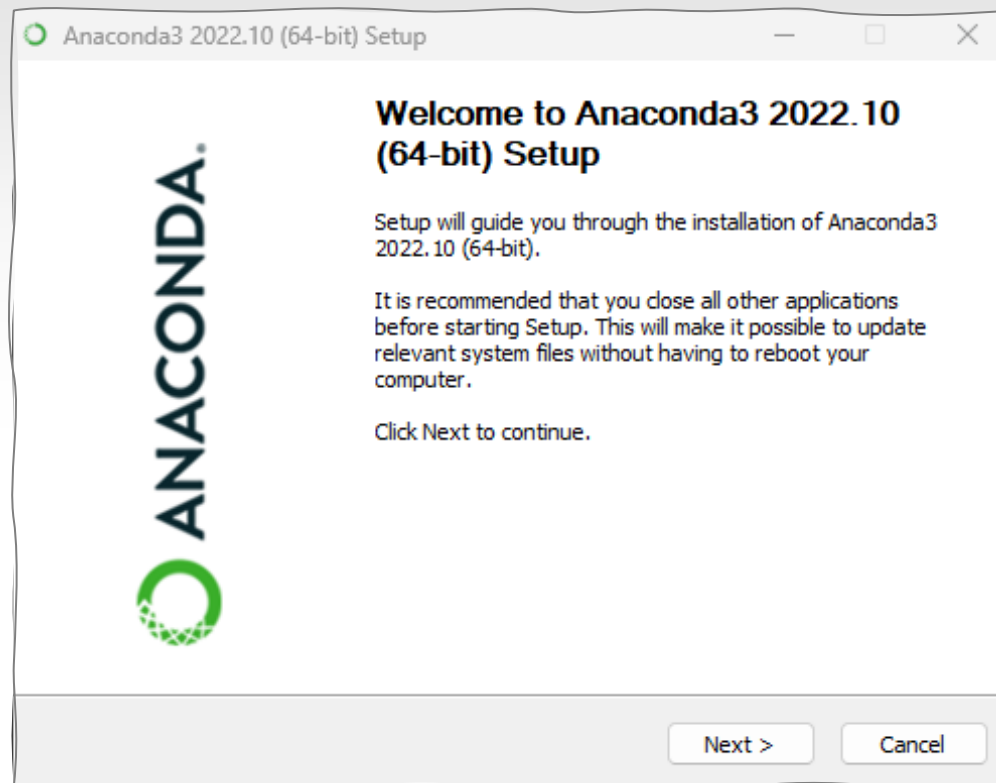


- **Instalação Anaconda**

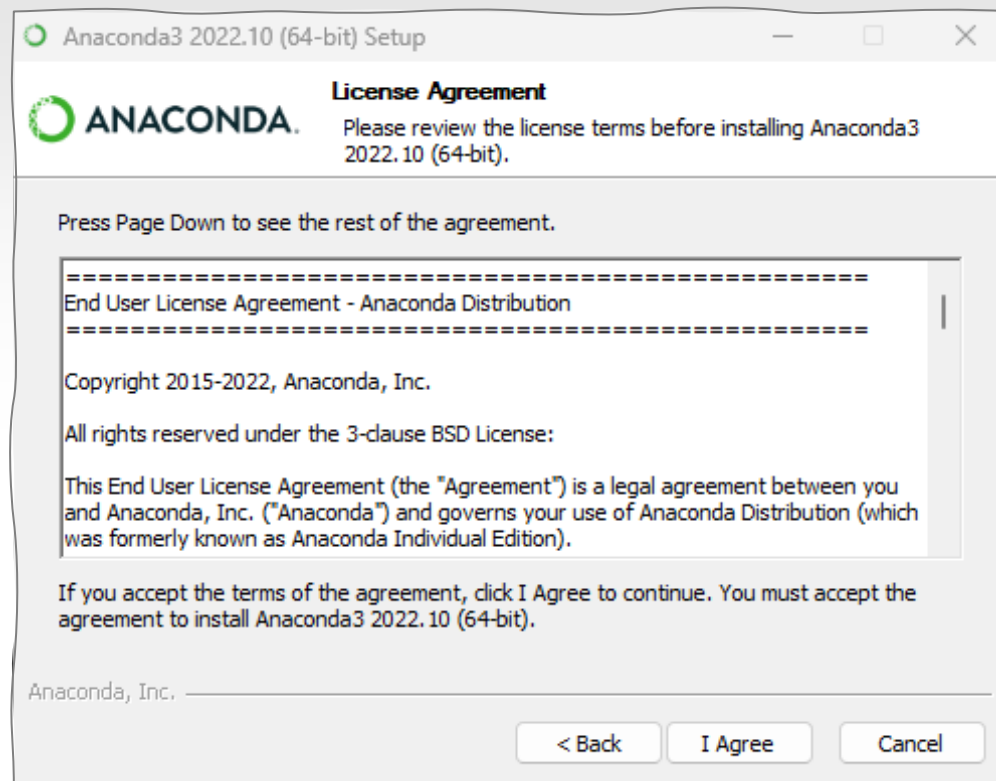
- **Download:** <https://repo.anaconda.com/archive/>



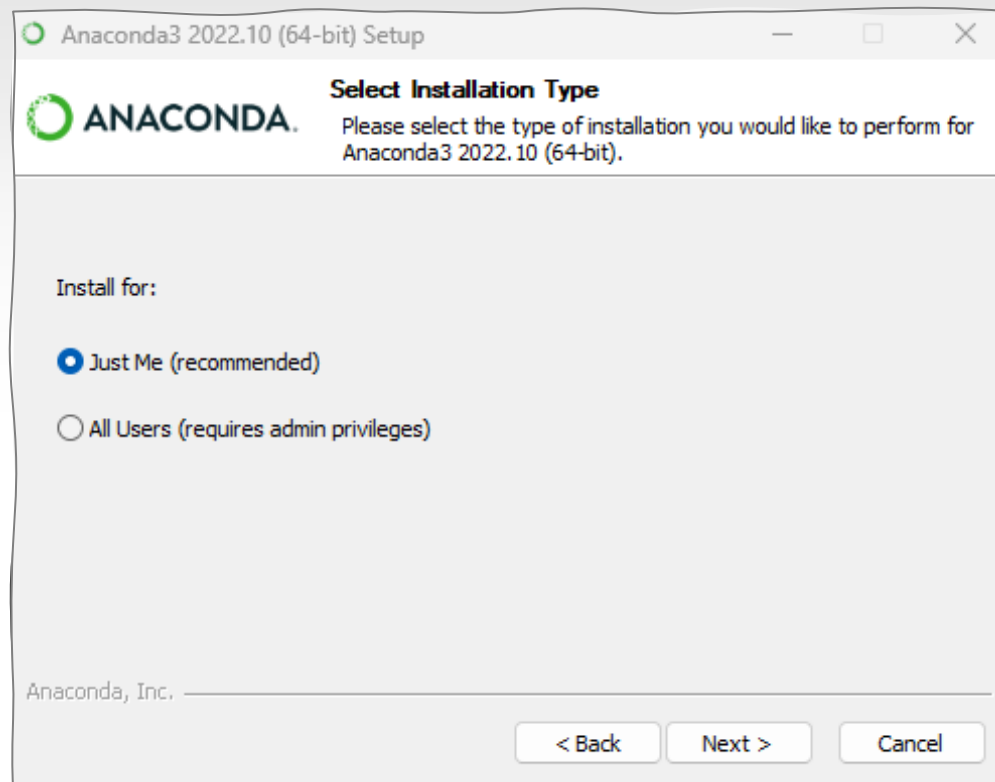
- ***Instalação Anaconda***
 - ***Welcome***



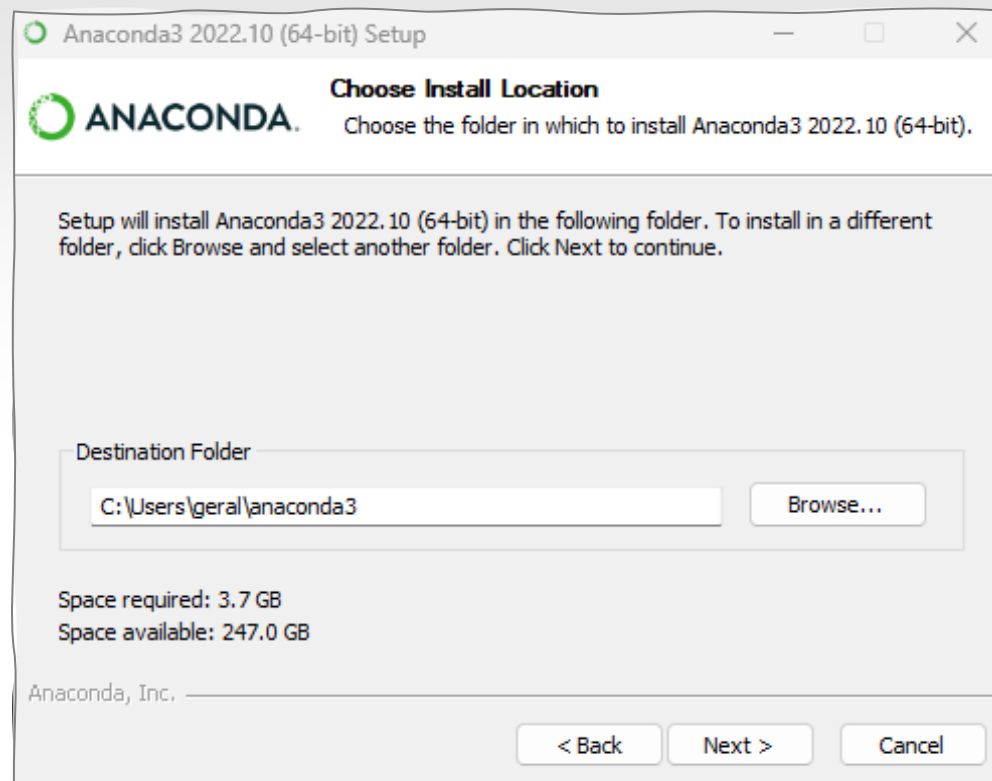
- ***Instalação Anaconda***
 - ***License Agreement***



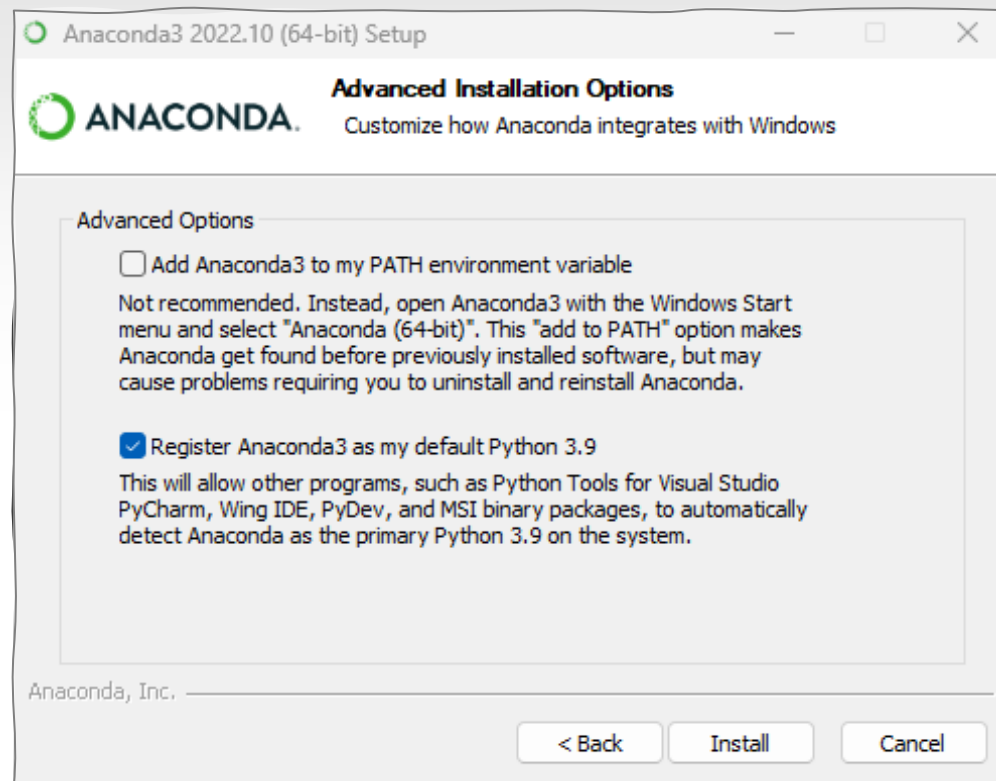
- ***Instalação Anaconda***
 - ***Installation Type***



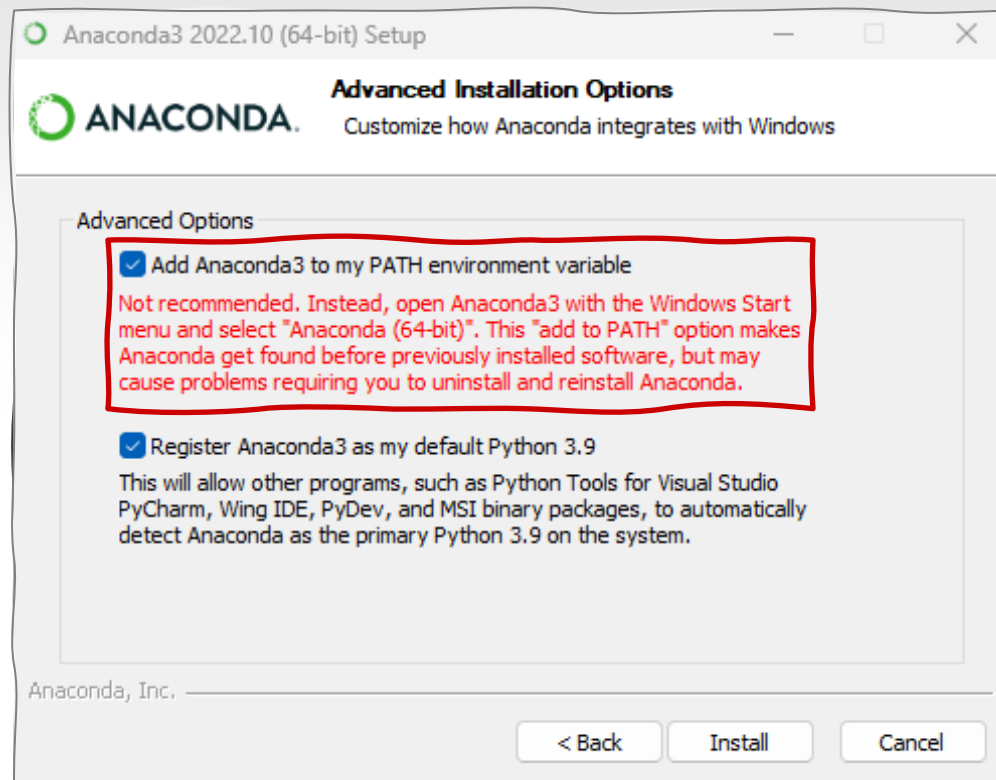
- ***Instalação Anaconda***
 - ***Install Location***



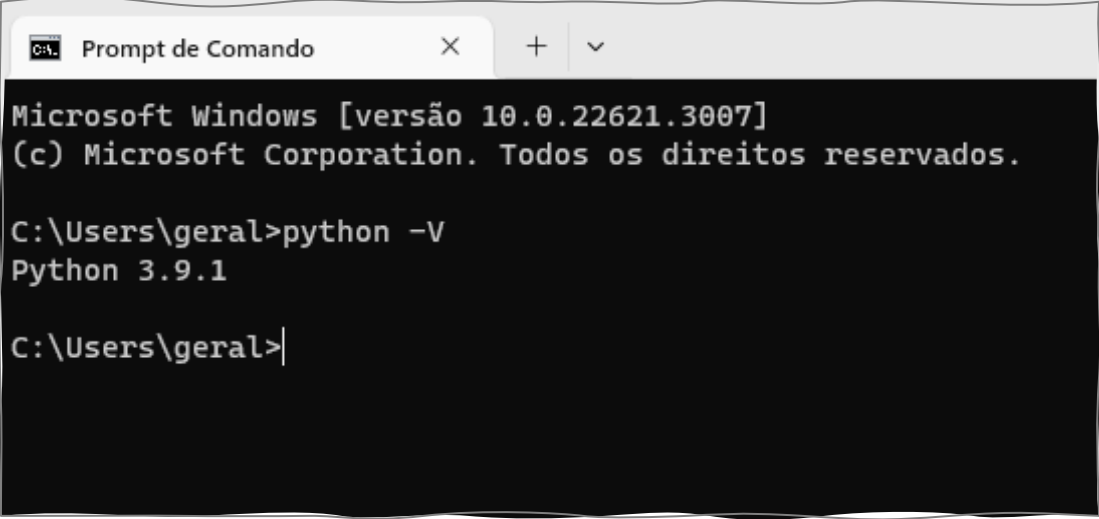
- ***Instalação Anaconda***
 - ***Installation Options***



- **Instalação Anaconda**
 - ***PATH Environment Variable***



- *Preparando o Ambiente*
 - *Verificando o êxito da instalação (Anaconda)*

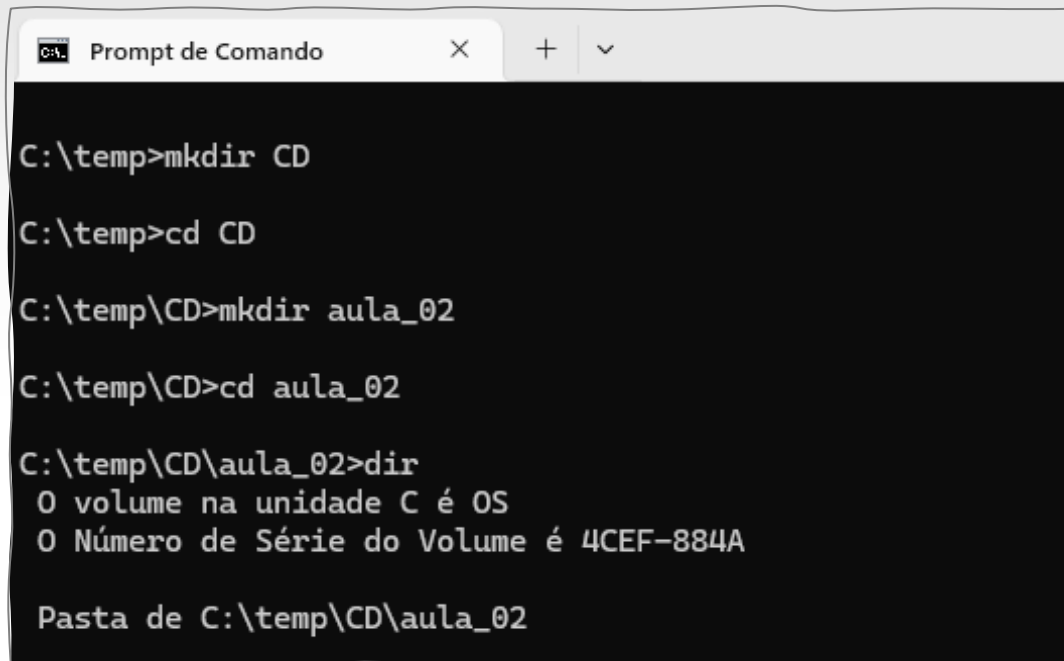


```
Microsoft Windows [versão 10.0.22621.3007]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\geral>python -V
Python 3.9.1

C:\Users\geral>|
```


- *Preparando o Ambiente*
 - Criando o diretório **CD** no "*c:\temp*"



```

C:\temp>mkdir CD

C:\temp>cd CD

C:\temp\CD>mkdir aula_02

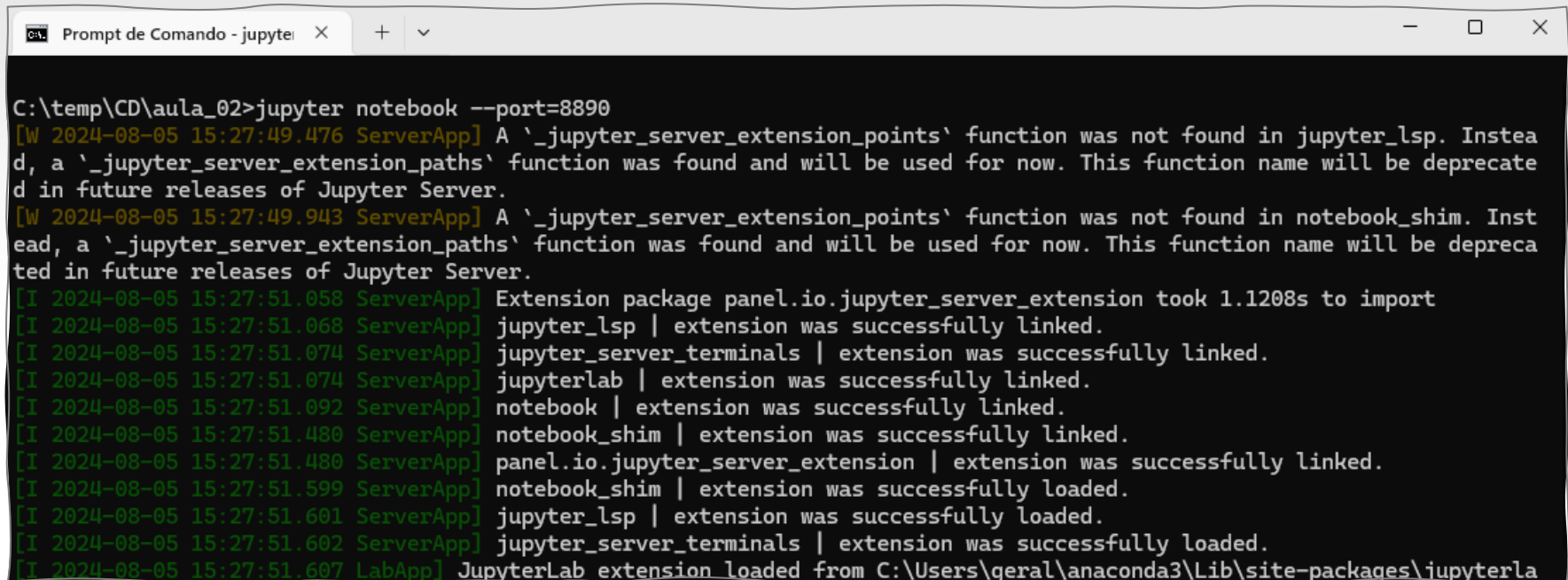
C:\temp\CD>cd aula_02

C:\temp\CD\aula_02>dir
O volume na unidade C é OS
O Número de Série do Volume é 4CEF-884A

Pasta de C:\temp\CD\aula_02

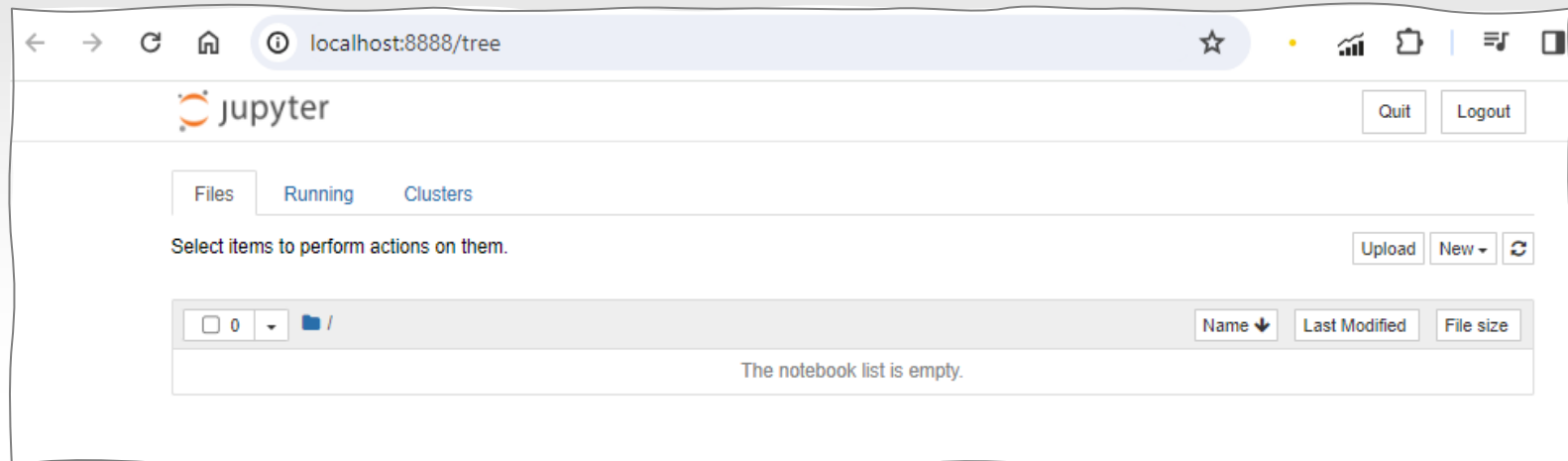
```

- *Preparando o Ambiente*
 - *Acessando o Jupyter Notebook*

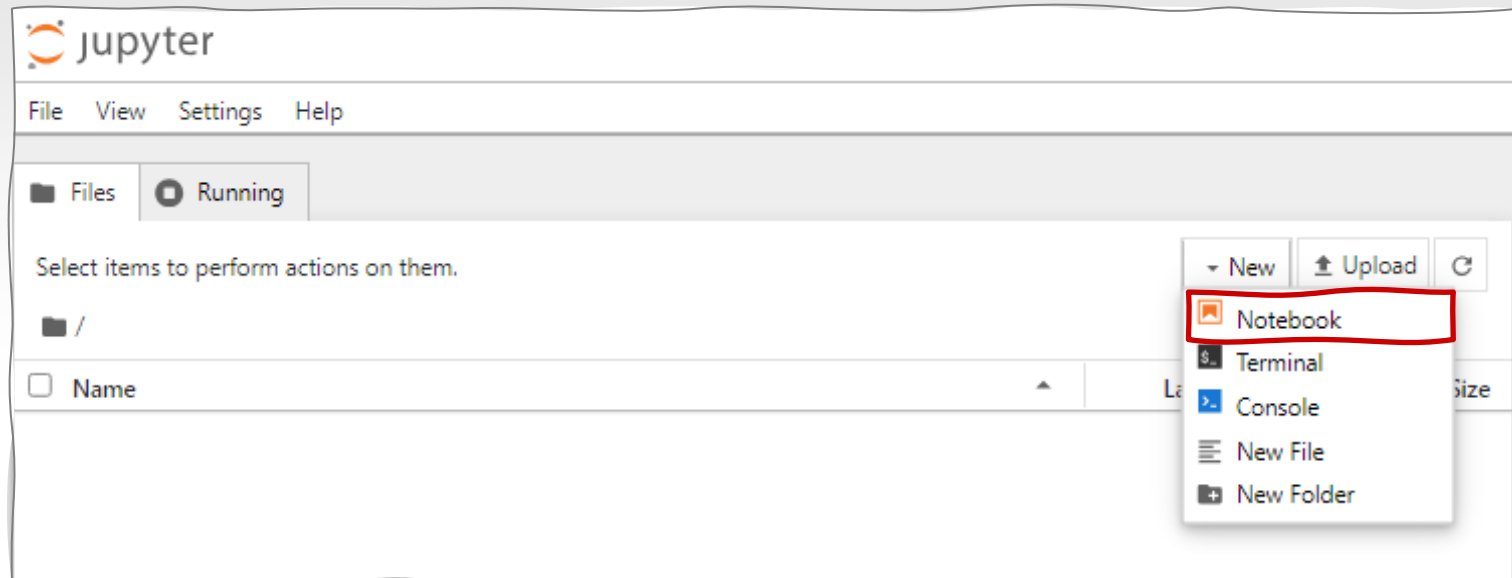


```
C:\temp\CD\aula_02>jupyter notebook --port=8890
[W 2024-08-05 15:27:49.476 ServerApp] A `_jupyter_server_extension_points` function was not found in jupyter_lsp. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[W 2024-08-05 15:27:49.943 ServerApp] A `_jupyter_server_extension_points` function was not found in notebook_shim. Instead, a `_jupyter_server_extension_paths` function was found and will be used for now. This function name will be deprecated in future releases of Jupyter Server.
[I 2024-08-05 15:27:51.058 ServerApp] Extension package panel.io.jupyter_server_extension took 1.1208s to import
[I 2024-08-05 15:27:51.068 ServerApp] jupyter_lsp | extension was successfully linked.
[I 2024-08-05 15:27:51.074 ServerApp] jupyter_server_terminals | extension was successfully linked.
[I 2024-08-05 15:27:51.074 ServerApp] jupyterlab | extension was successfully linked.
[I 2024-08-05 15:27:51.092 ServerApp] notebook | extension was successfully linked.
[I 2024-08-05 15:27:51.480 ServerApp] notebook_shim | extension was successfully linked.
[I 2024-08-05 15:27:51.480 ServerApp] panel.io.jupyter_server_extension | extension was successfully linked.
[I 2024-08-05 15:27:51.599 ServerApp] notebook_shim | extension was successfully loaded.
[I 2024-08-05 15:27:51.601 ServerApp] jupyter_lsp | extension was successfully loaded.
[I 2024-08-05 15:27:51.602 ServerApp] jupyter_server_terminals | extension was successfully loaded.
[I 2024-08-05 15:27:51.607 LabApp] JupyterLab extension loaded from C:\Users\geral\anaconda3\Lib\site-packages\jupyterlab
```

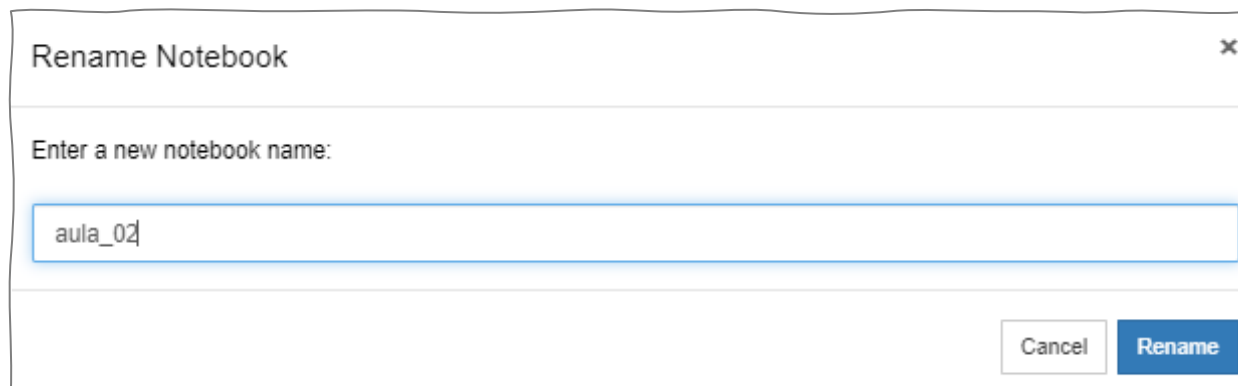
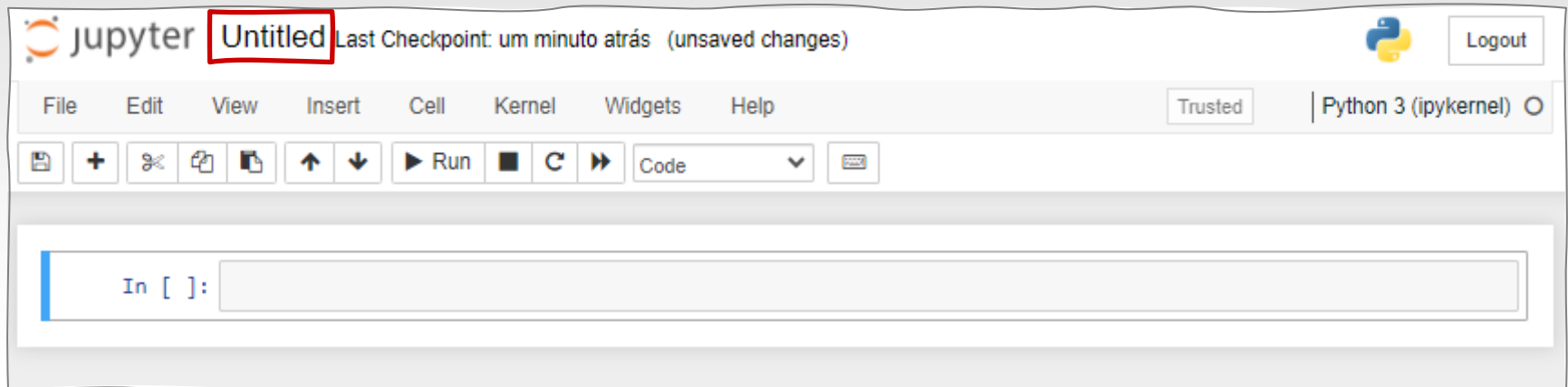
- ***Preparando o Ambiente***
 - ***Interface do Jupyter Notebook***

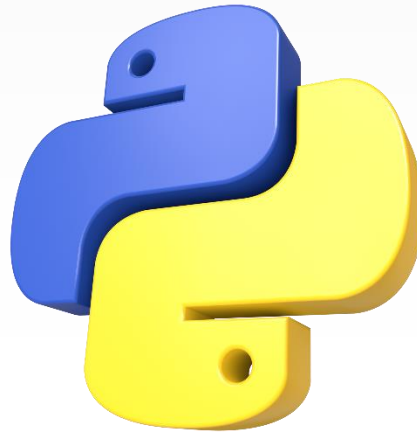


- ***Preparando o Ambiente***
 - ***Criando o primeiro Notebook***



- **Preparando o Ambiente**
 - **Nomeando o Notebook** (*aula_02*)





NÚMEROS E STRINGS

- **Números**
 - *Em programas de computador, quase sempre vamos manipular números*
 - *Um dos primeiros passos para aprender uma linguagem de programação é saber como operar seus números*
 - *Cada linguagem disponibiliza, por padrão, alguns tipos de números embutidos que estão presentes em sua sintaxe; há outros em módulos podem ser importados e são manipulados por meio de objetos e APIs*

- **Números**
 - *Python 3* possui **três tipos** de números embutidos: **int**, **float** e **complex**
 - **Exemplo (1):**
 - *Formas literais de escrever os números básicos em Python*

```
In [1]: int(1.0)
```

```
Out[1]: 1
```

```
In [2]: int('9')
```

```
Out[2]: 9
```

```
In [3]: float(1)
```

```
Out[3]: 1.0
```


- ***Números***
 - ***Exemplo (2): - Parte 1***
 - ***Manipulação numérica simples***

```
In [4]: 3 + 2
```

```
Out[4]: 5
```

```
In [5]: 3 + 4.2
```

```
Out[5]: 7.2
```

```
In [6]: 4 / 2
```

```
Out[6]: 2.0
```

```
In [7]: 5 / 2
```

```
Out[7]: 2.5
```

- **Números**
 - **Exemplo (2): - Parte 2**
 - **Manipulação de dados**

```
[1]: valor_int = 3          # Tipo inteiro
     valor_float = 1.5      # Tipo ponto flutuante
     string = "Olá Mundo!" # Tipo string
     val_bool = True        # Tipo booleano

     print(type(valor_int))
     print(type(valor_float))
     print(type(string))
     print(type(val_bool))

     <class 'int'>
     <class 'float'>
     <class 'str'>
     <class 'bool'>
```

- ***Números***
 - ***Exemplo (2): - Parte 3***
 - ***Manipulação de dados***

```
[2]: valor_int = 15
     valor_float = 1.5

     soma = valor_int + valor_float

     print(soma)
     print(type(soma))

16.5
<class 'float'>
```

- **Números**
 - **Exemplo (2): - Parte 4**
 - **Manipulação de dados**

```
[5]: # Conversão de um valor do tipo inteiro para um valor do tipo string
a = 100
b = str(a)

print(b)
print(type(b))

# Conversão de um valor do tipo float para um valor do tipo inteiro
c = 12.0
d = int(c)

print(d)
print(type(d))

# Conversão de um valor do tipo inteiro para um valor do tipo float
e = 19
f = float(e)

print(f)
print(type(f))
```

```
100
<class 'str'>
12
<class 'int'>
19.0
<class 'float'>
```

- **Operadores Aritméticos**

- *Os operadores aritméticos com números em Python são:*

- $x + y$ (*adição*)
 - $x - y$ (*subtração*)
 - x / y (*divisão em ponto flutuante*)
 - $x // y$ (*divisão descartando parte fracionária*)
 - $x * y$ (*multiplicação*)
 - $x \% y$ (*resto*)
 - $-x$ (*negação*)
 - $x ** y$ (*potência*)

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 1***
 - ***Utilizando expressões aritméticas e lógicas***

```
In [8]: 1 + 2
```

```
Out[8]: 3
```

```
In [9]: 3 - 1
```

```
Out[9]: 2
```

```
In [10]: 10 / 2
```

```
Out[10]: 5.0
```

```
In [11]: 10 // 3
```

```
Out[11]: 3
```

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 2***
 - ***Utilizando expressões aritméticas e lógicas***

```
In [12]: 10 * 2 + 1
```

```
Out[12]: 21
```

```
In [13]: 10 % 3
```

```
Out[13]: 1
```

```
In [14]: 2 ** 8
```

```
Out[14]: 256
```

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 3***
 - ***Utilizando expressões aritméticas e lógicas***

```
a = 10
b = 3

# Soma do valor de duas variáveis
soma = a + b

# Subtração do valor de duas variáveis
sub = a - b

# Divisão do valor de duas variáveis
div = a / b

# Multiplicação de valor de duas variáveis
mult = a * b
```

```
print('Soma: %d' % (soma))
print('Subtração: %d' % (sub))
print('Divisão: %d' % (div))
print('Multiplicação: %d' % (mult))
```

```
Soma: 13
Subtração: 7
Divisão: 3
Multiplicação: 30
```


- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 4***
 - ***Utilizando expressões aritméticas e lógicas***

```
# Resto de uma divisão de um valor do tipo inteiro por outro valor do tipo inteiro
res = a % b

# Resultado de elevar um valor do tipo inteiro a outro valor do tipo inteiro
exp = a ** b

# Piso, parte inteira de uma divisão de valores do tipo inteiro
pis = a // b

print('Resto: %d' % (res))
print('Exponencial: %d' % (exp))
print('Piso da divisão: %d' % (pis))

Resto: 1
Exponencial: 1000
Piso da divisão: 3
```

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 5***
 - ***Utilizando expressões aritméticas e lógicas***

```
nome = 'Joaquim'
idade = 38
altura = 1.80

print('Meu nome é %s, tenho %d anos e %.2f de altura!' % (nome, idade, altura))

Meu nome é Joaquim, tenho 38 anos e 1.80 de altura!
```

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 6***
 - ***Utilizando expressões aritméticas e lógicas***

```
aux_01 = 2
aux_02 = 3

print('O valor da primeira variável é {} enquanto o da segunda é {}'.format(aux_01, aux_02))
```

O valor da primeira variável é 2 enquanto o da segunda é 3

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 7***
 - ***Utilizando expressões aritméticas e lógicas***

```
a = 5
b = 2

c = a < b
d = a > b
e = a == b

print('Valor de c:', c)
print('Valor de d:', d)
print('Valor de e:', e)
print(a >= b)
print(b <= b)
```

```
Valor de c: False
Valor de d: True
Valor de e: False
True
True
```

- ***Operadores Aritméticos***
 - ***Exemplo (3): - Parte 8***
 - ***Utilizando expressões aritméticas e lógicas***

```
a = 10
b = 2

a += b # a + b
print(a)

a -= b # a - b
print(a)

a /= b # a / b
print(a)

a *= b # a * b
print(a)
```

```
12
10
5.0
10.0
```

- ***Operadores Aritméticos***

- ***Exemplo (3): - Parte 9***

- ***Calculando o índice de massa corpórea (IMC)***

```
nome = input('Qual é o seu nome: ')
idade = int(input('Qual sua idade: '))
altura = float(input('Qual sua altura: '))
peso = float(input('Qual seu peso: '))

print('Nome: %s' % nome)
print('Idade: %d' % idade)

imc = peso / (altura * altura)

print('O IMC =', imc)
print('Muito abaixo do peso: ', imc < 17)
print('Abaixo do peso normal: ', imc >= 17 and imc < 18.5)
print('Peso dentro do normal: ', imc >= 18.5 and imc < 25.0)
print('Acima do peso normal: ', imc >= 25 and imc < 30)
print('Muito acima do peso: ', imc >= 30)
```

```
Qual é o seu nome: Rafael de Souza
Qual sua idade: 45
Qual sua altura: 1.79
Qual seu peso: 79.8
Nome: Rafael de Souza
Idade: 45
O IMC = 24.90558971317999
Muito abaixo do peso: False
Abaixo do peso normal: False
Peso dentro do normal: True
Acima do peso normal: False
Muito acima do peso: False
```

Números

- *Mesclando Operações com Tipos Diferentes*
 - *Muitos programas podem operar **números** de **tipos diferentes** em uma mesma expressão*
 - *Podemos definir o **valor** de um **imposto** com um **float** e aplicar a um valor inteiro*
 - *Exemplo (4):*

```
In [15]: 100 * 1.3 # preço mais 30%  
Out[15]: 130.0
```

- ***Mesclando Operações com Tipos Diferentes***
 - ***Exemplo (5):***
 - ***Explorando diversas operações***

```
In [16]: type(1 + 2.0)
```

```
Out[16]: float
```

```
In [17]: type(1 + 2j)
```

```
Out[17]: complex
```

```
In [18]: type(1.0 + 2j)
```

```
Out[18]: complex
```

```
In [19]: type(1.0 + 1.0)
```

```
Out[19]: float
```


- **Função Type()**
 - *Utilizado para verificar o tipo de uma variável*
 - *Recebe apenas um **parâmetro** e retorna o **tipo** (ou **classe**) de um objeto*
 - *Para realizar testes em **condicionais**, é recomendado o uso da função **isinstance(obj, class)***



- **Manipulando Strings**

- Uma **string** é um **tipo** (ou **classe**) fundamental em muitas linguagens (nas de alto nível, sua manipulação geralmente é fácil)
- Python possui muitas conveniências na manipulação de strings
- Diferente de linguagens como C, **não** existe o tipo **char** (que é um tipo inteiro, de 1 byte, usado para representar valores da tabela ASCII)

Strings

- **Manipulando Strings**

- Em *Python*, existem **apenas strings**, mesmo que **contenham apenas um caractere ou sejam vazias**
- Podemos considerar que elas são **sequências** de **caracteres** de **tamanho** até o **máximo suportado**

"Python3" ← String
0123456 ← Indices
-7...-2-1 ← Negative Indexing

- ***Criando e Manipulando Strings***
 - ***Python tem formas muito convenientes de declarar strings e de formatá-las***
 - ***Exemplo (6):***

```
# coding: utf-8
```

```
"Copa 2022"
```

```
'Copa do mundo 2022'
```

```
"""2022 - Copa do mundo  
"""
```

```
"copa 'padrão fifa'"
```

```
'copa "padrão fifa"'
```

- ***Criando e Manipulando Strings***
 - ***Python tem formas muito convenientes de declarar strings e de formatá-las***
 - ***Exemplo (6):***

```
# coding: utf-8
```

```
"Copa 2022"
```

```
'Copa do mundo'
```

```
"""2022 - Copa  
do mundo"""
```

```
"copa 'padrão fifa'"
```

```
'copa "padrão fifa"'
```

Os formatos principais são com `'` e `"`, sendo que ao usar um, o outro pode ser usado internamente como nos dois últimos exemplos.

Strings

- ***Criando e Manipulando Strings***
 - ***Outra opção são as **multiline strings**, com **três aspas simples** ou **três aspas duplas** (formatação de saídas de console)***
 - ***Exemplo (7):***

```
In [20]: # coding: UTF-8

print("""\
    Uso: consulta_base[OPCOES]
    -h Exibe saída de ajuda
    -U url Url do dataset
""")

    Uso: consulta_base[OPCOES]
    -h Exibe saída de ajuda
    -U url Url do dataset
```

- ***Criando e Manipulando Strings***

- ***As strings literais separadas apenas por espaço serão implicitamente convertidas em uma única string literal***

```
("Copa" "2022") == "Copa2022"
```

- ***É possível quebrar strings longas que não são multiline apenas ao quebrar a linha, de preferência indentando-as***

```
input('Em qual cidade o legado da Copa foi mais relevante '  
      'para a população?')
```

Strings

- ***Criando e Manipulando Strings***
 - *String é uma sequência*
 - *Em strings podemos acessar os elementos **code points** usando um índice e a notação **variavel [index]***
 - *O índice varia de **0** (zero) até o tamanho da string **-1***
 - *Se ele for **negativo**, a contagem é na ordem inversa*

```
+---+---+---+---+---+---+
| P | y | t | h | o | n |
+---+---+---+---+---+---+
0   1   2   3   4   5   6
-6  -5  -4  -3  -2  -1
```


- ***Criando e Manipulando Strings***
 - *O termo sequência tem um significado especial*
 - *Para que um **objeto** seja uma **sequência**, como uma string, ele deve atender alguns requisitos*
 - *Três das suas **manipulações fundamentais** são: **saber tamanho**, **acessar um item** por **posição** e **acessar trechos** por **posições***
 - *Obter o **tamanho**, usamos **len**(string)*
 - *Acessar pelo **índice**, utilizamos **variavel**[índice]*
 - *Acessar fragmentos, usamos a **slice notation**, como em **minha_str**[1:2]*

- *Criando e Manipulando Strings*
 - *Exemplo (8):*

```
In [21]: str = "maracana"  
         str[0]
```

```
Out[21]: 'm'
```

```
In [22]: str[1:4]
```

```
Out[22]: 'ara'
```

```
In [23]: str[2:]
```

```
Out[23]: 'racana'
```

```
In [24]: str[:3]
```

```
Out[24]: 'mar'
```

```
In [25]: len(str)
```

```
Out[25]: 8
```

Strings

- ***Criando e Manipulando Strings***

- *Por ser uma sequência, além do acesso por índices e slices, podemos executar outras operações como:*

- $x \text{ in } y$, se x está em y
 - $x \text{ not in } y$, se x **não** está em y
 - $x + y$, concatenação de x com y
 - $x * y$, y repetições de x



- *Criando e Manipulando Strings*
 - *Exemplo (9):*

```
In [26]: "m" in "maracana"
Out[26]: True

In [27]: "x" not in "maracana"
Out[27]: True

In [28]: "m" + "aracana"
Out[28]: 'maracana'

In [29]: "a" * 3
Out[29]: 'aaa'
```

- **Imutabilidade**

- As **strings** são **sequências imutáveis** de **code points**
- Isso **significa** que elas **têm seu valor definido na criação** e que as **novas sempre são criadas a partir das operações com elas**
- **Exemplo (10):**
 - Se **tentarmos mudar o valor de uma posição** ou **pedaço** de uma **string**, vamos **receber um erro**

```
In [30]: minha_str = "disciplina python 3"
         minha_str[13] = "2"

-----
TypeError                                 Traceback (most recent call last)
Input In [30], in <module>
      1 minha_str = "disciplina python 3"
----> 2 minha_str[13] = "2"

TypeError: 'str' object does not support item assignment
```

- **Imutabilidade**
 - **Exemplo (11):**

```
In [31]: minha_str = "disciplina python 3"  
         minha_str = minha_str[0:18] + "2"  
         minha_str
```

```
Out[31]: 'disciplina python 2'
```

```
In [32]: minha_str = "disciplina python 3"  
         minha_str = minha_str.replace("3", "2")  
         minha_str
```

```
Out[32]: 'disciplina python 2'
```

- **Métodos**
 - Principais **métodos** para **manipulação** de **strings**:
 - **capitalize**
 - **count**
 - **endswith**
 - **join**
 - **split**
 - **startswith**
 - **replace**

- **Métodos**
 - **Exemplo (12):**

```
In [33]: "maracana".capitalize()
```

```
Out[33]: 'Maracana'
```

```
In [34]: "maracana".count("a")
```

```
Out[34]: 4
```

```
In [35]: "maracana".startswith("m")
```

```
Out[35]: True
```

```
In [36]: "maracana".endswith("z")
```

```
Out[36]: False
```


- **Métodos**
 - **Exemplo (13):**

```
In [37]: "copa de 2022".split(" ")
```

```
Out[37]: ['copa', 'de', '2022']
```

```
In [38]: " ".join(["Copa", "de", "2022"])
```

```
Out[38]: 'Copa de 2022'
```

```
In [39]: "copa de 2022".replace("2022", "2026")
```

```
Out[39]: 'copa de 2026'
```

- **Interpolação de Strings**

- Podemos **interpolare strings** usando **%** ou a função **.format()**
- Também implementa o formato **printf** de impressão
- **Exemplo (14):**

```
In [40]: "%d dias para copa" % (100)
```

```
Out[40]: '100 dias para copa'
```

```
In [41]: "{} dias para copa".format(100)
```

```
Out[41]: '100 dias para copa'
```

```
In [42]: "{dias} dias para copa".format(dias=100)
```

```
Out[42]: '100 dias para copa'
```

- *Interpolação de Strings*
 - Podemos configurar *espaçamentos* e *alinhamentos*
 - *Exemplo (14):*

```
In [43]: '{:<60}'.format('alinhado à esquerda, ocupando 60 posições')
```

```
Out[43]: 'alinhado à esquerda, ocupando 60 posições'
```

```
In [44]: '{:>60}'.format('alinhado à direita, ocupando 60 posições')
```

```
Out[44]: 'alinhado à direita, ocupando 60 posições'
```

```
In [45]: '{:^60}'.format('centralizado, ocupando 60 posições')
```

```
Out[45]: 'centralizado, ocupando 60 posições'
```

```
In [46]: '{:.^60}'.format('centralizando ao alterar caractere de espaçamento')
```

```
Out[46]: '.....centralizando ao alterar caractere de espaçamento.....'
```

- *Criando e Manipulando Strings*
 - *Todas as strings são representações em unicode code points*
 - *Podem ter caracteres com acentos (por exemplo, "é") ou qualquer outro unicode válido, como caracteres do japonês*
 - *A questão é que muitos caracteres, inclusive da língua portuguesa, não podem ser representados em apenas 1 byte*

- ***Criando e Manipulando Strings***

- *Dessa forma, foi necessária uma **diferenciação** entre os **objetos** que utilizam um **byte por letra**, que seriam apenas as **letras** e **caracteres** da **tabela ASCII**, e os **objetos** que usam caracteres que necessitam de mais de **1 byte** para serem representados*
- *No **Python 2**, uma **string** é uma sequência de **bytes**, logo **não** podem representar unicodes a não ser que estes sejam convertidos para **bytes** com um **codec**. Nessa versão, muitas vezes uma **conversão implícita** era feita usando **UTF-8**, e **strings** e **bytes** são o mesmo objeto, enquanto **Unicode** é outro*

- ***Criando e Manipulando Strings***
 - No **Python 3**, **strings** e **unicodes** são a mesma coisa (*string*) e os **bytes** um **outro objeto**
 - No **Python 2**, a **string** "é" é implicitamente convertida para sua **representação** em **bytes**, usando **UTF-8**
 - Nesse caso, o seu **tamanho** seria **len("é") == 2** (no **Python 3.3**, a mesma **string** teria **tamanho 1**, porque ela contém apenas um **code point** e representa isso semanticamente)
 - Logo, o tamanho seria **len("é") == 1**

- ***Criando e Manipulando Strings***

- *Isso poderia representar uma perda de espaço se os caracteres ASCII (os que conseguem ser representados em 1 byte) tivessem que ocupar 2 bytes em memória*
- *A PEP-393 (Flexible String Representation) faz com que Python 3 represente os code points de maneira eficiente em memória*
- *Isso deve-se ao fato de que um unicode pode ser convertido para bytes por meio de diversos codecs, sendo os mais famosos o UTF-8 e o UTF-16*
- *O mesmo unicode torna-se uma sequência de bytes diferente, dependendo do codec usado na conversão*

- **Comandos Simples**

- Realizam uma **ação específica**, como **atribuição** de valor, **chamada** de uma função ou **operação** de leitura/escrita
- Uma sequência de comandos **simples** com mesma **indentação** forma um comando **composto**
- Comandos de controle definem como será a execução de um **bloco** de comandos podendo ser **condicionais** ou de **repetição** (iterativos)

- **Comando if**
 - **Pode ser entendido** como “*se uma condição é verdadeira, um bloco de comandos é executado*”
 - **Sintaxe:**

```
if (expressão condicional):  
    bloco de comandos
```

- **Comando *if***
 - **Exemplo (15):**
 - **Utilizando a estrutura condicional *if***

```
nota = int(input('Digite uma nota: '))  
  
if nota < 60:  
    print('Reprovado')  
else:  
    print('Aprovado')
```

```
Digite uma nota: 70  
Aprovado
```

- **Comando if**
 - **Exemplo (16):**
 - **Utilizando a estrutura condicional if**

```
nota = int(input('Digite uma nota: '))

if nota < 30:
    print('Reprovado')
elif nota <= 50:
    print('Recuperação')
else:
    print('Aprovado')
```

```
Digite uma nota: 70
Aprovado
```

- **Comando if**
 - **Exemplo (17):**
 - **Utilizando a estrutura condicional if**

```
n1 = int(input('Digite o primeiro número: '))
n2 = int(input('Digite o segundo número: '))

if n1 > n2:
    print(n1, ' maior que ', n2)
elif n1 < n2:
    print('%d maior que %d' % (n2, n1))
else:
    print('Os números são iguais')
```

```
Digite o primeiro número: 10
Digite o segundo número: 20
20 maior que 10
```

- **Comando while**

- *Executa um bloco de comandos repetidamente enquanto uma dada condição for verdadeira, ou seja, até uma condição se tornar falsa*

- **Sintaxe:**

```
while (expressão condicional):  
    bloco de comandos
```

- **Comando while**
 - **Exemplo (18):**
 - **Utilizando a estrutura de repetição while**

```
contador = 1

while contador <= 5:
    print(contador)
    contador += 1
```

```
1
2
3
4
5
```

- **Comando while**
 - **Exemplo (19):**
 - **Utilizando a estrutura de repetição while**

```
nota = -1

while nota < 0 or nota > 10:
    nota = int(input('Digite uma nota válida de 0 a 10: '))

print('Nota: %s' % nota)
```

Digite uma nota válida de 0 a 10: 11
Digite uma nota válida de 0 a 10: -2
Digite uma nota válida de 0 a 10: 10
Nota: 10

- **Comando for**
 - **Utilizado quando sabemos quantas vezes o bloco de comandos que ele controla deve ser executado**
 - **O número de vezes pode ser definido pela função *range*, que estabelece uma sequência de valores**
 - **Sintaxe:**

```
for (número de iterações):  
    bloco de comandos
```


- ***Comando while***
 - ***Exemplo (20):***
 - ***Utilizando a estrutura de repetição com range***

```
print(list(range(5)))  
print(list(range(1, 5)))  
print(list(range(2, 10, 3)))
```

```
[0, 1, 2, 3, 4]  
[1, 2, 3, 4]  
[2, 5, 8]
```

- **Comando *while***
 - **Exemplo (21):**
 - **Utilizando a estrutura de repetição com *for***

```
for i in range(5):  
    print('Bem-vindo a disciplina de Ciência de Dados!')
```

```
Bem-vindo a disciplina de Ciência de Dados!  
Bem-vindo a disciplina de Ciência de Dados!  
Bem-vindo a disciplina de Ciência de Dados!  
Bem-vindo a disciplina de Ciência de Dados!  
Bem-vindo a disciplina de Ciência de Dados!
```

Repetição

- **Comando while**
 - **Exemplo (22):**
 - **Utilizando a estrutura de repetição *for***

```
for i in range(0, 5):  
    print(i)
```

```
0  
1  
2  
3  
4
```

- ***Comando while***
 - ***Exemplo (23):***
 - ***Varia de 0 a 10, incrementando de 2 em 2***

```
for i in range(0, 10, 2):  
    print(i)
```

```
0  
2  
4  
6  
8
```

Repetição

- **Comando while**
 - **Exemplo (24):**
 - **Utilizando a estrutura de repetição *for***

```
total = 0

for i in range(0, 5):
    num = int(input('Digite um num: '))
    total += num

print('Soma: %s' % (total))
print('Média: %s' % (total/5))

Digite um num: 5
Digite um num: 6
Digite um num: 7
Digite um num: 8
Digite um num: 9
Soma: 35
Média: 7.0
```

Repetição

- **Comando while**
 - **Exemplo (25):**
 - **Utilizando a estrutura de repetição *for***

```
tab = int(input('Tabuada do número: '))  
  
for i in range(10):  
    print('%d x %d = %d' % (tab, i + 1, tab * (i + 1)))
```

```
Tabuada do número: 5  
5 x 1 = 5  
5 x 2 = 10  
5 x 3 = 15  
5 x 4 = 20  
5 x 5 = 25  
5 x 6 = 30  
5 x 7 = 35  
5 x 8 = 40  
5 x 9 = 45  
5 x 10 = 50
```

- ***Estruturas de Dados***

- São um dos **principais componentes** de qualquer **linguagem de programação**, por oferecer opções **computacionalmente eficientes**, em **tempo** de **execução** e uso de **memória**, para **armazenar e processar dados**
- As **estruturas de dados** **podem seguir duas abordagens**:
 - **Estruturas lineares**: **organizam seus itens de forma independente de seus conteúdos**
 - **Exemplo**:
 - **Estrutura de listas**

- ***Estruturas de Dados***
 - *As estruturas de dados podem seguir duas abordagens:*
 - ***Estruturas associativas:*** *permitem acesso a seus itens de forma independente de sua posição na memória do dispositivo computacional*
 - ***Exemplos:***
 - *Estruturas chave*
 - *Valor (key, value)*
 - *Árvores*
 - *Tabelas hash*

- ***Estruturas de Dados***
 - *A linguagem Python disponibiliza estruturas de dados **predefinidas**, dentre elas existem as listas (**list**), conjuntos (**set**), tuplas (**tuples**) e dicionários (**dictionary**)*
 - *Listas e tuplas são utilizadas para armazenar sequências*
 - *Principais características das sequências:*
 - *Verificar se itens presentes em sequências, fazendo testes de pertinência com os operadores **in** (pertence) e **not in** (não pertence)*

- ***Estruturas de Dados***
 - ***Principais características das sequências:***
 - ***Realizar operações de indexação, que permitem acesso direto a um item de uma sequência, utilizando o operador de subscrição ([posição do item])***
 - ***Extrair uma fatia de uma sequência, por meio de operadores de fatiamento***

- *Estruturas de Dados*

- *Cada estrutura de dados utiliza uma forma de **armazenamento** e facilita determinado conjunto de operações para manipulação de dados*
- *As estruturas de dados diferem em:*
 - **Mutabilidade:** *se mutáveis, os itens podem ser modificados, alterando conteúdo e tamanho. Se forem imutáveis, seus itens não podem ser modificados*
 - **Ordem:** *se a posição de um item pode ser usada para acessar o seu valor*

- ***Estruturas de Dados***
 - ***Exemplo (26):***
 - ***Gerando uma lista vazia***

```
# Criando uma lista vazia
lista = []

print(lista)

[]
```

O índice (posição) dos itens começa com o valor 0 (zero), e cada novo item terá como índice o valor inteiro que segue o índice do item imediatamente anterior. Dessa forma, o índice do último item em uma lista de n itens será $n - 1$.

Valores podem ser de qualquer tipo, e tipos podem estar misturados.

- ***Estruturas de Dados***
 - ***Exemplo (27):***
 - ***Gerando diversas listas***

```
lista_num = [1, 2, 3]
lista_nomes = ['André', 'Roberto', 'Arthur']
lista_misturada = [37, 'Roberto', 'Arthur', 9, 0, 'Liliane', 2]

print(lista_num)
print(lista_nomes)
print(lista_misturada)

[1, 2, 3]
['André', 'Roberto', 'Arthur']
[37, 'Roberto', 'Arthur', 9, 0, 'Liliane', 2]
```

- ***Estruturas de Dados***
 - ***Exemplo (28):***
 - ***Acessando itens da lista***

```
for item in lista_num:  
    print(item)
```

```
print(lista_nomes[0])  
print(lista_nomes[1])
```

```
1  
2  
3  
André  
Roberto
```

- **Listas**
 - ***Lista inclui funções para:***
 - ***consultar (procurar) itens em uma lista***
 - ***alterar tanto a composição de uma lista como a ordem dos itens em uma lista***
 - ***adicionar itens a uma lista***
 - ***remover itens de uma lista***

- **Listas**
 - **Exemplo de funções aplicadas a uma lista:**
 - ***len(): retorna o tamanho de uma lista***
 - ***append(): adiciona um item ao final de uma lista***
 - ***pop(): remove o item que está na última posição de uma lista***
 - ***pop([i]): remove o item na i-ésima posição de uma lista***
 - ***remove(x): remove de uma lista um item cujo valor é igual a “x”***
 - ***insert(x): insere um item de valor “x” na i-ésima posição de uma lista***

- **Listas**
 - **Exemplo de funções aplicadas a uma lista:**
 - ***clear(): remove todos os itens de uma lista***
 - ***count(): conta o número de vezes que o valor “x” aparece***
 - ***sort(): ordena os itens de uma lista***
 - ***index(x): retorna o índice do primeiro valor igual a “x”***

- **Listas**
 - **Exemplo (29):**
 - **Manipulando itens de uma lista**

```
lista = [1, 2, 3, 3, 3, 4, 5, 6, 7]

print('Tamanho da lista: %d' % len(lista))

lista.append(8)
print(lista)

lista.pop()
print(lista)

lista.pop(0)
print(lista)

lista.remove(2)
print(lista)
```

```
Tamanho da lista: 9
[1, 2, 3, 3, 3, 4, 5, 6, 7, 8]
[1, 2, 3, 3, 3, 4, 5, 6, 7]
[2, 3, 3, 3, 4, 5, 6, 7]
[3, 3, 3, 4, 5, 6, 7]
```

- **Listas**
 - **Exemplo (30):**
 - **Manipulando itens de uma lista**

```
lista.insert(0, 1) # Posição 0 (zero), item 1
print(lista)

print(lista.count(3))

print(lista.index(1))

lista.clear()
print(lista)
```

```
[1, 3, 3, 3, 4, 5, 6, 7]
3
0
[]
```

- **Listas**
 - **Exemplo (31):**
 - **Manipulando itens de uma lista**

```
lista = [1, 2, 3, 4, 5]

print(lista[::-1]) # Inverte a ordem dos itens em uma lista

print(lista[0:2]) # Retorna os valores que estão nas posições do intervalo

print(lista[-1])  # Retorna o último item da lista

lista.sort()      # Ordenar

print(lista)

lista[0] = 2      # Alterar item

print(lista)

[5, 4, 3, 2, 1]
[1, 2]
5
[1, 2, 3, 4, 5]
[2, 2, 3, 4, 5]
```

- ***Tuplas***

- *Semelhante à estrutura de dados listas, com a diferença de ser uma **estrutura imutável***
- *Quando um item é adicionado a uma tupla, o mesmo **não** poderá ser **alterado***
- *As **tuplas** são criadas com itens separados por vírgulas e entre **parênteses***

- ***Tuplas***

- ***Exemplo (32):***

- ***Criando uma tupla com 4 elementos***

```
tupla_num = (1, 2, 3, 4)
print(tupla_num)
(1, 2, 3, 4)
```

- ***Tentando alterar algum valor na tupla***

```
# Tentando alterar a estrutura (tupla_num)
tupla_num[0] = 2

-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 2
      1 # Tentando alterar a estrutura (tupla_num)
----> 2 tupla_num[0] = 2

TypeError: 'tuple' object does not support item assignment
```

- ***Tuplas***
 - ***Exemplo (33):***
 - ***Manipulando elementos da tupla***

```
# Manipulando elementos da tupla

print(tupla_num.count(1)) # contando a quantidade de elementos "1"
print(tupla_num.index(2)) # apresentando o índice do elemento "2"
print(len(tupla_num))     # contando o total de elementos

1
1
4
```

- ***Tuplas***

- ***Exemplo (34):***

- ***Para usar algumas funções da lista ([append](#), [pop](#)), torna-se necessário realizar a conversão da tupla para uma lista***

```
# Convertendo tupla em lista
tupla_conv = list(tupla_num)

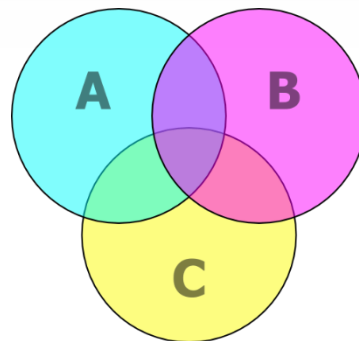
# Adicionando um item na última posição
tupla_conv.append(5)
print("Elemento 5 adicionado: %s" % (tupla_conv))

# Removendo o item adicionado
tupla_conv.pop()
print("Elemento 5 removido: %s" % (tupla_conv))

Elemento 5 adicionado: [1, 2, 3, 4, 5]
Elemento 5 removido: [1, 2, 3, 4]
```


- **Conjuntos**

- Um **conjunto** em **Python** é uma **coleção** de **itens únicos** que **não** seguem uma **ordem específica** (**sem elementos repetidos**)
- É uma **estrutura de dados mutável**
- **Permite a eliminação de itens duplicados**



- **Conjuntos**
 - **Exemplo (35):**
 - **Eliminação dos elementos repetidos**

```
frutas = ['maçã', 'laranja', 'maçã', 'pera', 'laranja', 'banana']  
  
frutas_n_duplicados = set(frutas)  
  
print(frutas_n_duplicados)  
  
{'pera', 'banana', 'maçã', 'laranja'}
```

- **Conjuntos**
 - A linguagem Python disponibiliza **operadores** para implementar as operações usadas em teoria de conjuntos
 - **pertinência de um item ao conjunto**
 - **união** entre conjuntos
 - **interseção** entre conjuntos
 - **diferença** entre conjuntos
 - **complemento de um conjunto**
 - **leis De Morgan**

- **Dicionários**

- Também chamado de **memória associativa** ou **vetor associativo**
- É útil para **consultar, incluir, alterar ou remover itens e seus significados, ou informações associadas**
- São indexados por **chaves** (keys) de um **tipo imutável**, que **utilizam a estrutura chave: valor** (key: value)
- São criados utilizando **chaves**, todavia, na **inserção de valores**, é feita por **colchetes**

- ***Dicionários***

- ***Exemplo (36):***

- ***Manipulando dicionários em Python***

```
# Criação de um dicionário com preço de frutas por kg
dic_frutas = {'Maçã': 5.50,
              'Banana': 2.50,
              'Pera': 6.50,
              'Uva': 7.20}

# Inserção de um novo item, fruta e seu preço, em um dicionário
dic_frutas['Manga'] = 6.20

print(dic_frutas)

{'Maçã': 5.5, 'Banana': 2.5, 'Pera': 6.5, 'Uva': 7.2, 'Manga': 6.2}
```

- **Dicionários**

- **Exemplo (37):**

- **Consultando alguns valores de um dicionário**

```
# Consulta do preço de uma fruta armazenada em um dicionário

print('Uva: %s' % dic_frutas['Uva'])
print('Pera: %s' % dic_frutas['Pera'])

Uva: 7.2
Pera: 6.5
```

- **Dicionários**

- **Exemplo (38): - Parte 1**

- *É possível construir dicionário de forma incremental, por meio de combinação de listas*
 - *Retornando o resultado de uma atleta em um competição por meio da média das distâncias cobertas em dois saltos*

```
# Criação de um dicionário com o desempenho de atletas
atletas = {}

# Atribuição de um nome fictício, início, a um dicionário
nome_atleta = 'início'

# Inclusão do nome de atletas em um dicionário
while nome_atleta != '':
    nome_atleta = input('Atleta: ')
    if nome_atleta != '':
        saltos = [] # Lista para armazenar a distância dos saltos
        for i in range(0, 2): # 2 saltos
            saltos.append(float(input('%s Salto: ' % (i + 1))))

        # Adicionando uma lista de saltos ao dicionário
        atletas[nome_atleta] = saltos
```

- **Dicionários**

- **Exemplo (38): - Parte 2**

- **Retornando o resultado de uma atleta em um competição por meio da média das distâncias cobertas em dois saltos**

```
# Percorrer um dicionário retornando seus itens
print('\nResultado Final:')
for nome_atleta, saltos in atletas.items():
    print('Atleta: %s' % nome_atleta)
    print('Saltos: ', str(saltos))
    print('Média dos Saltos: %.2f m' % (sum(saltos)/2))
```

```
Atleta: Rafael
1 Salto: 1.50
2 Salto: 1.60
Atleta: Pedro
1 Salto: 1.20
2 Salto: 1.10
Atleta: Joaquim
1 Salto: 1.60
2 Salto: 2.30
Atleta:

Resultado Final:
Atleta: Rafael
Saltos: [1.5, 1.6]
Média dos Saltos: 1.55 m
Atleta: Pedro
Saltos: [1.2, 1.1]
Média dos Saltos: 1.15 m
Atleta: Joaquim
Saltos: [1.6, 2.3]
Média dos Saltos: 1.95 m
```


- **Dicionários**

- **Exemplo (39):**

- Para **percorrer dicionários**, é **podemos utilizar a estrutura de repetição *for***

```
dic_frutas = {'Maçã': 5.50,  
              'Banana': 2.50,  
              'Pera': 6.50}  
  
for fruta, valor in dic_frutas.items():  
    print('Fruta: %s' % fruta)  
    print('Preço: %s' % valor)
```

```
Fruta: Maçã  
Preço: 5.5  
Fruta: Banana  
Preço: 2.5  
Fruta: Pera  
Preço: 6.5
```

- *Por que usar funções?*
 - *Em muitos programas, um bloco de código é executado **várias vezes** em outros fragmentos de código*
 - *Em vez de repetir esse bloco várias vezes ao longo do programa, tornando o código mais longo e menos claro, esses blocos podem ser escritos como **funções**, que podem ser chamadas sempre que o programa tiver que executá-lo*
 - *Quanto mais **complexa** a solução que um programa deve implementar, maior a chance dele usar funções para facilitar a escrita e entendimento do código*

- **Sintaxe:**

```
def nome_funcao(parametro_um, parametro_dois, ...):  
    comando  
    ...  
    comando  
    return dados
```

Muitos programas em Python utilizam funções, que são chamados de métodos quando declaradas no contexto de orientação de objetos.

- ***Funções***
 - ***Exemplo (40):***
 - ***Implementando a função soma***

```
# Trecho que define a função
def soma(x, y, z):
    return x + y + z

# Trecho em que a função é chamada
s = soma(10, 20, 40)
print(s)
```

70

- **Funções**
 - **Exemplo (41):**
 - **Implementando a função par/ímpar**

```
# Trecho que define a função
def par_impar(n):
    if n % 2 == 0:
        return True
    else:
        return False

# Trecho em que a função é chamada
print(par_impar(10))
print(par_impar(15))
```

```
True
False
```

- **Funções**

- **Exemplo (42):**

- **Implementando a função para calcular o IMC**

```
# Trecho que define a função
def imc(nome, idade, altura, peso):
    print('Nome: %s' % nome)
    print('Idade: %d' % idade)

    imc = peso / ( altura * altura )
    print('O IMC = ', imc)

    if imc < 17:
        print('Muito abaixo do peso')
    elif imc >= 17 and imc < 18.5:
        print('Abaixo do peso normal')
    elif imc >= 18.5 and imc < 25.0:
        print('Peso dentro do normal')
    elif imc >= 25 and imc < 30:
        print('Acima do peso normal')
    else:
        print('Muito acima do peso')

# Trecho em que a função é chamada
imc('Guilherme', 28, 1.85, 98)
```

```
Nome: Guilherme
Idade: 28
O IMC = 28.634039444850252
Acima do peso normal
```

- ***Funções de Entrada e Saída***

- ***A entrada de dados pode ser proveniente de várias fontes (de um teclado, de um arquivo ou de um microfone)***
- ***A saída também pode ser apresentada em vários meios (na tela de um dispositivo computacional, em uma folha de papel impressa por uma impressora ou em arquivo digital)***
- ***Eventualmente, torna-se necessário imprimir os resultados gerados em arquivos, para salvá-los para consultas futuras***
- ***O inverso também pode ocorrer, quando precisamos ler em arquivos os dados de entradas***

- ***Funções de Entrada e Saída***

- ***Funções para manipulação de arquivos:***

- ***open([nome_arquivo](#), 'r'):*** ***abre um arquivo no modo leitura***
 - ***open([nome_arquivo](#), 'w'):*** ***abre um arquivo no modo escrita ([início do documento](#))***
 - ***open([nome_arquivo](#), 'a'):*** ***abre um arquivo no modo escrita ([fim do documento](#))***
 - ***close():*** ***finaliza a utilização do arquivo***
 - ***write():*** ***adiciona uma nova linha no arquivo***
 - ***read():*** ***lê uma string ou o arquivo***
 - ***readline():*** ***retorna uma linha do arquivo***
 - ***readlines():*** ***retorna uma lista contendo cada linha do arquivo***

- ***Funções de Entrada e Saída***
 - ***Exemplo (43):***
 - ***Realizando a leitura de um arquivo***

```
doc = open('dados/doc.txt', 'r')  
  
for linha in doc.readlines():  
    print(linha)  
  
doc.close()
```

```
Nome,Matricula,TB1,TB2  
  
Rafael,01554,7.0,8.0  
  
Andre,01551,8.5,7.2  
  
Liliane,01441,7.0,8.6  
  
Victor,01335,9.2,5.0  
  
Cristiane,01245,7.0,9.0  
  
Joana,01412,8.0,7.5
```

- ***Funções de Entrada e Saída***
 - ***Exemplo (44):***
 - ***Realizando a leitura de um arquivo***

```
# Utilização da função split() para decompor uma string em uma lista
doc = open('dados/doc.txt', 'r')

# Decomposição da string separando componentes por vírgulas para gerar
# um arquivo do tipo "csv"

for linha in doc:
    valores = linha.split(',')
    print(valores[0], valores[1], valores[2], valores[3])
```

Nome Matricula TB1 TB2

Rafael 01554 7.0 8.0

Andre 01551 8.5 7.2

Liliane 01441 7.0 8.6

Victor 01335 9.2 5.0

Cristiane 01245 7.0 9.0

Joana 01412 8.0 7.5

- ***Funções de Entrada e Saída***
 - ***Exemplo (45):***
 - ***Adicionando uma nova linha no final arquivo***

```
doc = open('dados/doc.txt', 'a')

# Adicionando uma nova linha
doc.write('Roberto, ')
doc.write('01414, ')
doc.write('8.5, ')
doc.write('7.5 ')
doc.write("\n") # Quebra de linhas
doc.close()

# Leitura do arquivo com a nova linha
doc = open('dados/doc.txt', 'r')

for linha in doc.readlines():
    print(linha)

doc.close()
```

```
Nome,Matricula,TB1,TB2
Rafael,01554,7.0,8.0
Andre,01551,8.5,7.2
Liliane,01441,7.0,8.6
Victor,01335,9.2,5.0
Cristiane,01245,7.0,9.0
Joana,01412,8.0,7.5
Roberto, 01414, 8.5, 7.5
```

Referências

NETTO, A. MACIEL, F. Python Para Data Science: e Machine Learning Descomplicado, Alta Books, 2021.

BEHRMAN, K. R. Fundamentos de Python para Ciência de Dados. Bookman, 2022.

GRUS, J. Data Science do Zero - 2ª Edição: Noções Fundamentais com Python. Alta Books, 2021.

